

★ WEEK 14 · PART 1 of 3

Markov Chain Monte Carlo: Sample, Don't Integrate

The Engine That Made Bayes Practical

Monte Carlo · the histogram IS the posterior · why a Markov chain

Last week's catch was one intractable integral. MCMC sidesteps it entirely: instead of computing the posterior, we draw samples from it — and read every answer off the draws.

1

PART 1 · SAMPLE, DON'T INTEGRATE

Turning Integrals Into Averages

The integral that blocked us, Monte Carlo turning it into an average of draws, the histogram as the posterior, and why a Markov chain is the way to generate those draws.

Born at Los Alamos — Part 1

An algorithm to dodge an impossible integral

In the late 1940s, physicists at Los Alamos faced integrals no one could solve. Stanislaw Ulam and John von Neumann answered with Monte Carlo — estimate the answer by random sampling — a method named for the famous casino.

The problem they faced

- Physics calculations needed impossible integrals.
- No formula, and grids were hopeless in high dimensions.
- They needed a completely different approach.

The Monte Carlo answer

- Estimate the integral by random sampling.
- Averaging draws approximates the quantity.
- The 1953 Metropolis paper made it a chain.

Born at Los Alamos — Part 2

From physics to the Bayesian revolution

The Metropolis method sat mostly in physics for decades. Hastings generalized it in 1970, and in the 1980s and 90s statisticians realized it was the key to computing posteriors — sparking the modern Bayesian revolution.

The long incubation

- For years it was a physicist's tool.
- Hastings generalized the algorithm in 1970.
- Its statistical power went unnoticed for a while.

The revolution

- It was exactly what Bayesian inference needed.
- Suddenly, real posteriors became computable.
- It is why Bayes is everywhere today.

The Integral That Blocked Us

Where Week 13 left off

1

Bayes needs a normalizer

The posterior requires dividing by the evidence — an integral of the likelihood times the prior over all parameters.

2

That integral is usually intractable

For any realistic model it has no closed form, and in many dimensions it can't be done numerically either.

3

MCMC sidesteps it entirely

Instead of computing the posterior, we draw samples from it — and every summary we want is just a summary of those draws.

Part 1 in one line: don't compute the posterior — sample it, and read the answers off the draws.

The Problem We Are Solving

One integral stood in the way

Everything hinges on the normalizing integral. Where conjugacy gave it for free, real models don't. Without it, the posterior is known only up to a constant — enough to evaluate, but not, it seems, to use.

What we can do

- Evaluate the likelihood times the prior at any point.
- So we know the posterior's SHAPE.
- We just don't know its scale (the normalizer).

What we can't do

- Compute that normalizing integral directly.
- So we can't turn the shape into a proper distribution.
- And we can't sample from it the easy way.

Monte Carlo: Integrals Become Averages

The move that changes everything

$$\mathbb{E}_p[f(\theta)] = \int f(\theta) p(\theta) d\theta \approx \frac{1}{N} \sum_{i=1}^N f(\theta_i), \quad \theta_i \sim p$$

The identity

- An expectation is an integral against a distribution.
- It also equals the average of samples from that distribution.
- So sampling replaces integrating.

Why it helps

- Averages are easy; high-dimensional integrals are not.
- The error shrinks predictably with more draws.
- It scales to dimensions where grids fail completely.

From Integrals to Averages

A hard analytic problem becomes an easy numerical one

The old way

- Set up and solve an integral analytically.
- Or grid the space and sum — hopeless in high dimensions.
- Both fail for realistic posteriors.

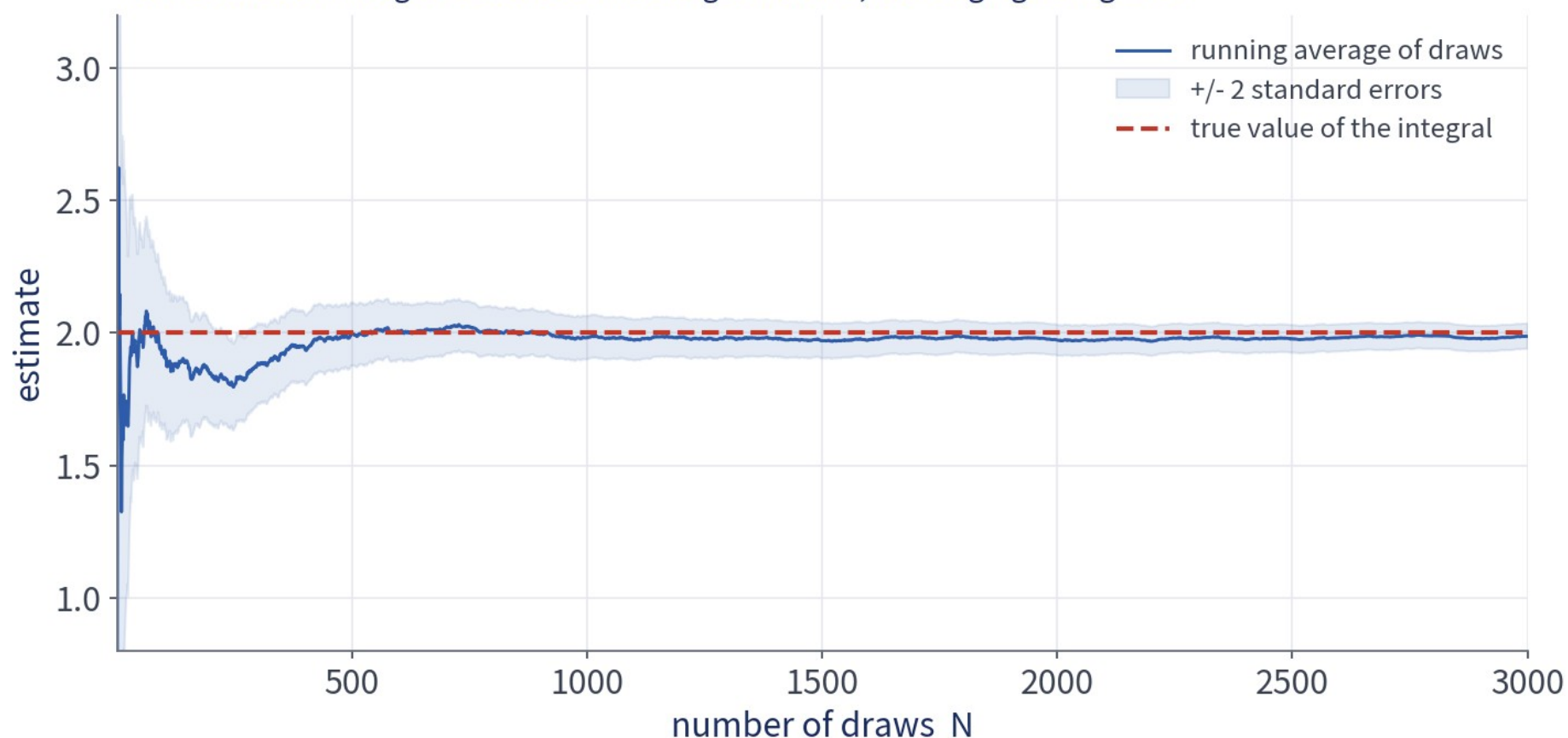
The Monte Carlo way

- Draw samples from the distribution.
- Average the quantity of interest over them.
- Accuracy improves with the square root of the draws.

An Integral, Estimated by Averaging

The running average of draws converges to the true value of the integral, and its error band narrows as the number of draws grows. Integration has become simple averaging.

Monte Carlo: an integral becomes an average of draws, converging as N grows



Sample, Then Summarize

The whole plan in four words

The plan is simple: get a big pile of draws from the posterior, then compute whatever you want as a summary of those draws. Every Bayesian quantity — means, intervals, forecasts — is just a statistic of the samples.

Step one: sample

- Generate many draws from the posterior.
- Each draw is a plausible parameter value.
- Together they represent the whole distribution.

Step two: summarize

- The mean of the draws is the posterior mean.
- Their quantiles give a credible interval.
- Their spread is the uncertainty.

From Samples to Answers

Everything you want is a summary of draws

Point summaries

- Posterior mean: average the draws.
- Posterior median or mode: read off the draws.
- Any function of the parameter: transform each draw.

Uncertainty and prediction

- Credible interval: take sample quantiles.
- Probability of a claim: fraction of draws satisfying it.
- Predictive: simulate new data from each draw.

The Histogram IS the Posterior

Collect draws; read every answer off them

There is no separate 'posterior' to compute — the histogram of your draws IS the posterior, as far as any practical question is concerned. With enough samples, the histogram matches the true density to any precision you like.

What this means

- The draws stand in for the distribution itself.
- Any summary of the density is a summary of the draws.
- You never need the closed-form density.

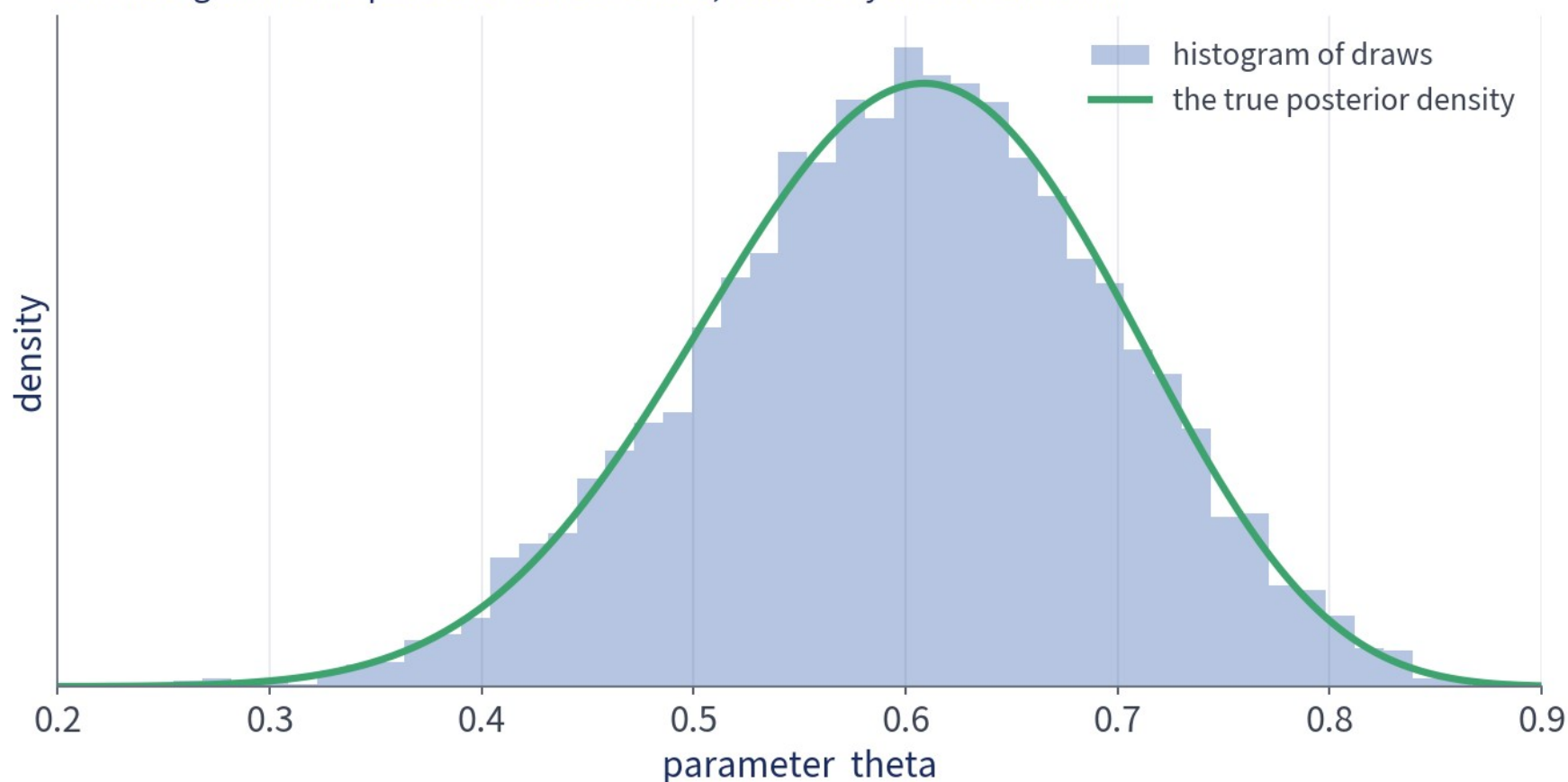
The reassurance

- More draws, a closer match to the truth.
- The approximation error is controllable.
- It is exact in the limit of infinite draws.

The Histogram IS the Posterior

The histogram of a few thousand draws traces the true posterior density almost exactly. Once you have the draws, the density itself is redundant — the samples answer every question.

The histogram IS the posterior: collect draws, read every answer off them



But We Can't Sample It Directly

The posterior isn't a distribution we can just draw from

Why not

- We only know the posterior up to the normalizer.
- It is not a standard distribution with a built-in sampler.
- It lives in many dimensions with a complex shape.

The bind

- Monte Carlo needs draws from the posterior.
- But we can't draw from it independently.
- We need a cleverer way to generate samples.

We CAN Evaluate the Target

Even without the normalizer

$$\text{we CAN evaluate } p(\theta \mid \text{data}) \propto \underbrace{P(\text{data} \mid \theta) P(\theta)}_{\text{no normalizer needed}}$$

What we can compute

- At any parameter value, the likelihood times the prior.
- That is the posterior's shape, up to a constant.
- No integral is needed to evaluate it at a point.

Why this is enough

- The chain only ever compares two points.
- In a ratio, the unknown constant cancels.
- Pointwise evaluation is all MCMC requires.

Why a Markov Chain?

Build a walk that settles into the posterior

Since we can't draw from the posterior directly, we build a random walk that visits parameter values in proportion to the posterior. Let it run, and the values it visits become our samples.

The idea

- Construct a walk over the parameter space.
- Design it to linger where the posterior is high.
- Its long-run visits mirror the posterior.

Why a chain works

- Each step only needs the target at two points.
- So the normalizer never appears.
- The walk explores the whole distribution over time.

What a Markov Chain Is

We have met these before

The definition

- A sequence where the next step depends only on the current one.
- It is memoryless beyond the present (recall Week 10).
- Here the 'state' is a parameter value.

The new twist

- In Week 10 the chain was the model.
- Here the chain is a COMPUTATIONAL device.
- We design it so its long-run behavior is the posterior.

The Stationary Distribution

Make the walk's long-run law the posterior

design a chain whose stationary distribution is $\pi(\theta) = p(\theta \mid \text{data})$

The target

- Every chain has a long-run stationary distribution.
- We engineer the walk so that law IS the posterior.
- Then its visited values are posterior draws.

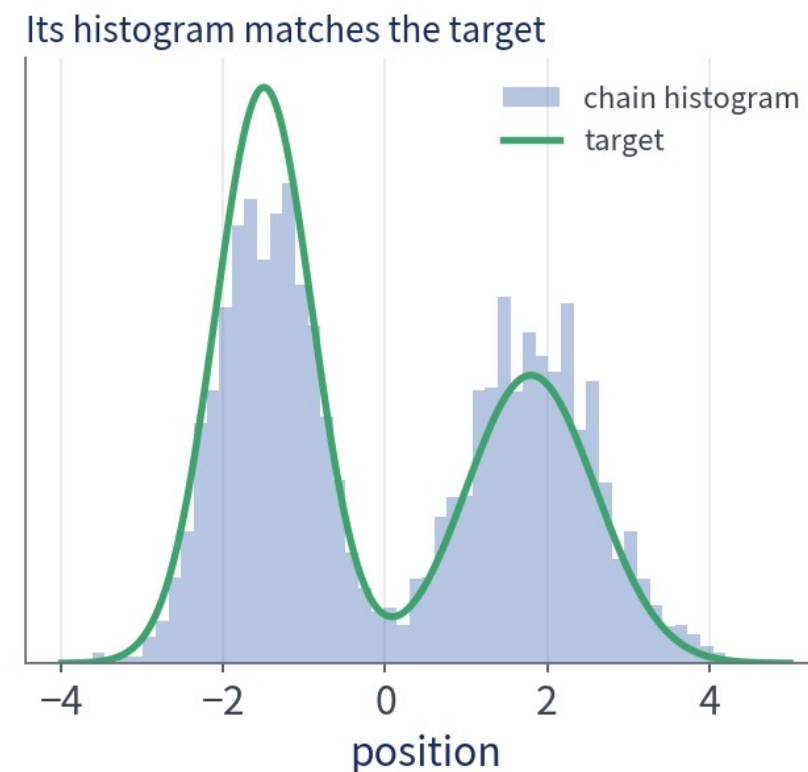
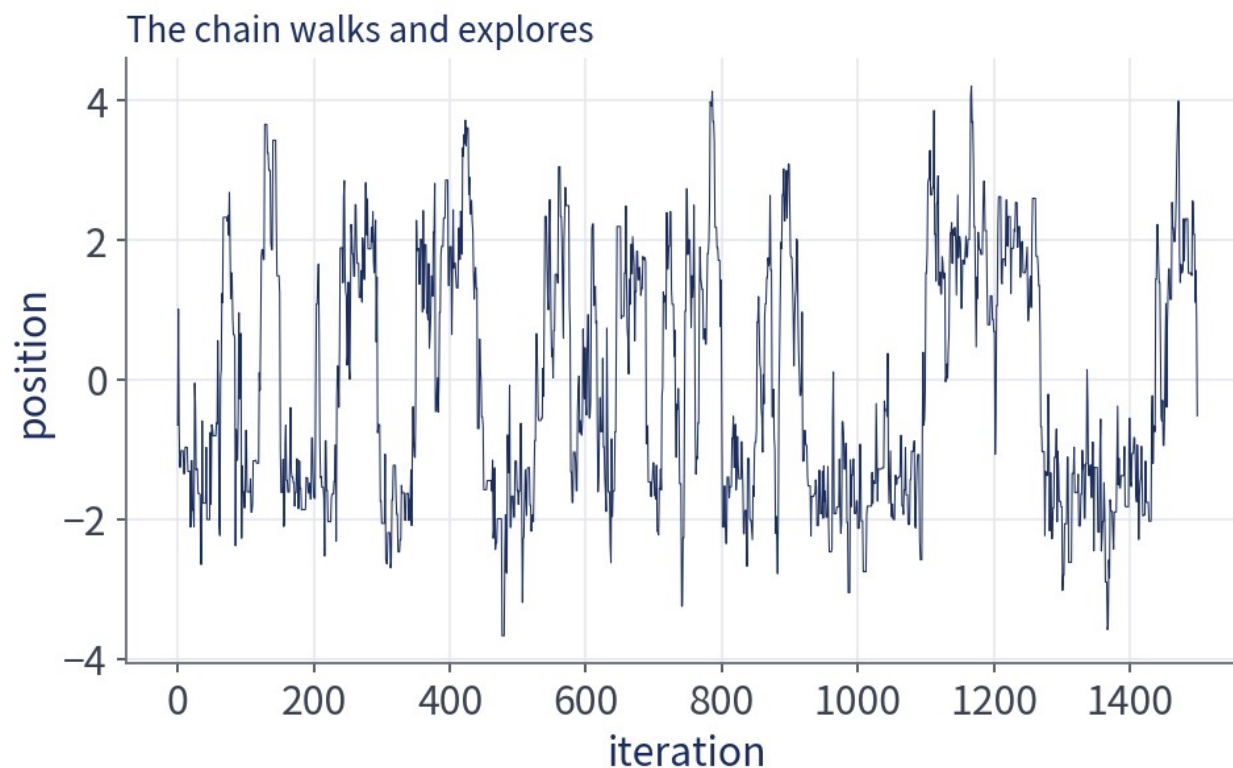
Why it works

- After it settles, positions are distributed as the posterior.
- Collect them, and you have your samples.
- The algorithms of Part 2 build exactly such walks.

A Chain Settling Into the Target

A well-designed walk explores the parameter space, and the histogram of where it goes matches the target distribution — even a two-humped one. The chain's visits become posterior samples.

A Markov chain, designed well, settles into the target distribution



Correlated but Valid Samples

MCMC draws are dependent — and that's okay

The dependence

- Each draw is a small step from the last.
- So consecutive samples are correlated, not independent.
- The chain wanders rather than jumping freely.

Why it still works

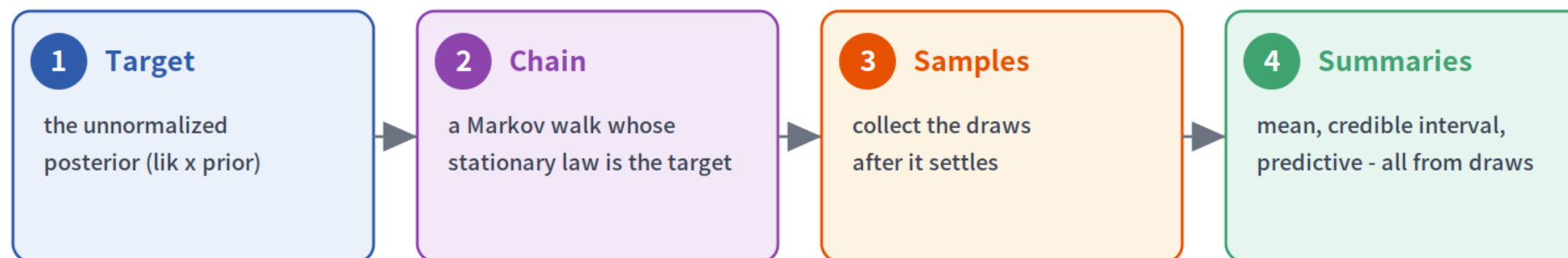
- Averages over the draws are still valid.
- You just need more draws for the same precision.
- Diagnostics (Part 3) tell you how many are enough.

The MCMC Recipe at a Glance

Take the unnormalized target, build a Markov chain whose stationary law is that target, collect the draws once it settles, and read all your summaries off them. Evaluate the target; never integrate it.

The MCMC Recipe at a Glance

Never compute the integral - build a walk that visits values in proportion to the posterior.



The whole trick in one line

You only ever need to EVALUATE the target (likelihood times prior) at points - never to integrate it.
The chain turns those pointwise evaluations into a stream of draws whose histogram is the posterior.

Sample, don't integrate - then read every answer off the draws.

What Parts 2 and 3 Add

From idea to working sampler

Part 2 — the algorithms

- Metropolis-Hastings: propose a move, accept or reject.
- Why the normalizer cancels in the acceptance rule.
- Gibbs sampling: update one parameter at a time.

Part 3 — the craft

- Trace plots and convergence diagnostics.
- Burn-in, mixing, and how many draws you need.
- Modern tools like PyMC, and the models we can now fit.

Monte Carlo Integration

Direct Claude Code to turn an integral into an average

GOAL See Monte Carlo in action: estimate a quantity by sampling, and watch the estimate converge.

[INPUT] Prompt · Claude Code

Write and run `monte_carlo.py` that:

1. picks a quantity with a known answer - e.g. estimate π by sampling points in a square, or an expectation under a known distribution;
 2. estimates it as the average over N random draws;
 3. plots the running estimate vs N with an error band, and the true value as a line;
 4. shows the error shrinking like one over root N .
- Confirm the estimate converges to the known answer.

How to work with it

- An integral becomes an average of draws.
- The estimate converges as N grows.
- Error falls like one over root N .
- This is the engine under all of MCMC.

Deliverable: `monte_carlo.py` + a running-estimate plot converging to the known value.

Three Questions Before You Move On

If you can answer these, Part 1 has landed

- | | | |
|-----------|---|-------------------------------------|
| Q1 | Monte Carlo replaces an integral with what? | An average of draws |
| <hr/> | | |
| Q2 | For MCMC, what do we need to compute at a point? | The target, up to a constant |
| <hr/> | | |
| Q3 | We design the Markov chain so its stationary law is what? | The posterior |

All three come from Monte Carlo, the unnormalized target, and the stationary-distribution design.

Three Misconceptions to Avoid

Where MCMC intuition slips

Common myths

- "MCMC computes the normalizing integral."
- "You need independent draws for valid answers."
- "MCMC finds the single best parameter."

The reality

- It sidesteps the integral — only ratios are needed.
- Correlated draws are fine; you just need more of them.
- It maps the WHOLE posterior, not one point.

Three Things to Carry Forward

Part 1 in one screen

1

Sample, don't integrate

Monte Carlo turns an intractable integral into an easy average of draws. Every posterior summary — mean, interval, forecast — becomes a summary of samples.

2

The histogram is the posterior

With enough draws, their histogram matches the posterior to any precision. We only ever need to EVALUATE the target (likelihood times prior), never to normalize it.

3

A Markov chain generates the draws

Since we can't sample the posterior directly, we build a walk whose stationary distribution IS the posterior. Its correlated-but-valid visits are our samples.

Next — Part 2: Metropolis-Hastings and Gibbs, the algorithms that build the chain.

★ WEEK 14 · PART 1 · PRACTICE

Sample, Don't Integrate - Worked by Hand

Answers as Averages of Draws, and How Many You Need

Monte Carlo estimation and the $1/\sqrt{N}$ law

Part 1's core move, with the arithmetic shown. We turn a posterior integral into an average of draws - reading off the mean and the probability of profit - then work out how many samples precision demands, and why correlated MCMC draws cost more.

1P

PART 1 · APPLICATIONS

Turn Integrals Into Averages

Case A: Monte Carlo estimation - the posterior mean and the probability of profit, each just a summary of draws.
Case B: convergence - the $1/\sqrt{N}$ law, the samples a target precision needs, and the price of correlated draws.
Then a Claude Code lab.

What Part 1 Told Us

Now we make it concrete - with numbers

1

An integral becomes an average

Monte Carlo replaces a hard posterior integral with an easy average over draws. That single move unblocks everything.

2

Every answer is a summary of draws

The mean, an interval, the probability of profit - all are just summaries of the same set of posterior samples.

3

Draws can be correlated but still valid

MCMC draws are dependent, so they carry less information each. You need more of them for the same precision.

Everything here is a calculator exercise: a handful of draws, a fraction, and a sample count.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from a set of posterior draws.
- Summarize them, or count how many you need.
- Then read the answer or the cost.

What you should watch

- How an expectation becomes an average.
- How a probability becomes a fraction.
- How the error falls as 1 over root- N .

Answers From Draws

The mean, and the probability of profit

Once you have posterior draws of a strategy's expected return, every question is a summary of them. The posterior mean is their average. The probability the strategy is profitable is the fraction of draws above zero. And the Monte Carlo error tells you how much to trust that fraction. We compute all three by hand

The draws

- A small set, to show the averaging.
- Then a full set of 1000 draws.
- Of the expected annual return.

What we'll do by hand

- Average the draws for the mean.
- Count the winners for $P(\text{profitable})$.
- Compute the Monte Carlo error.

Monte Carlo

Integrals become averages

$$\mathbb{E}[f(\theta)] = \int f(\theta) p(\theta | D) d\theta \approx \frac{1}{N} \sum_{i=1}^N f(\theta^{(i)}) \quad (\text{a hard integral becomes an easy average})$$

The move

- The expectation is an integral over the posterior.
- Replace it with an average over draws.
- The more draws, the better the average.

Why it works

- The sample average estimates the true mean.
- It converges as the draw count grows.
- Hard analysis becomes easy arithmetic.

The Posterior Mean

Just average the draws

$$\hat{\mathbb{E}}[\text{return}] = \frac{5+8+11+6+10}{5} = 8.0\% \quad (\text{the posterior mean is just the sample average})$$

The calculation

- Five draws: 5, 8, 11, 6, and 10 percent.
- Their sum is 40.
- Divided by five, the mean is 8%.

The point

- No integral was solved.
- The mean is a plain average.
- The same works for thousands of draws.

The Probability of Profit

Count the winning draws

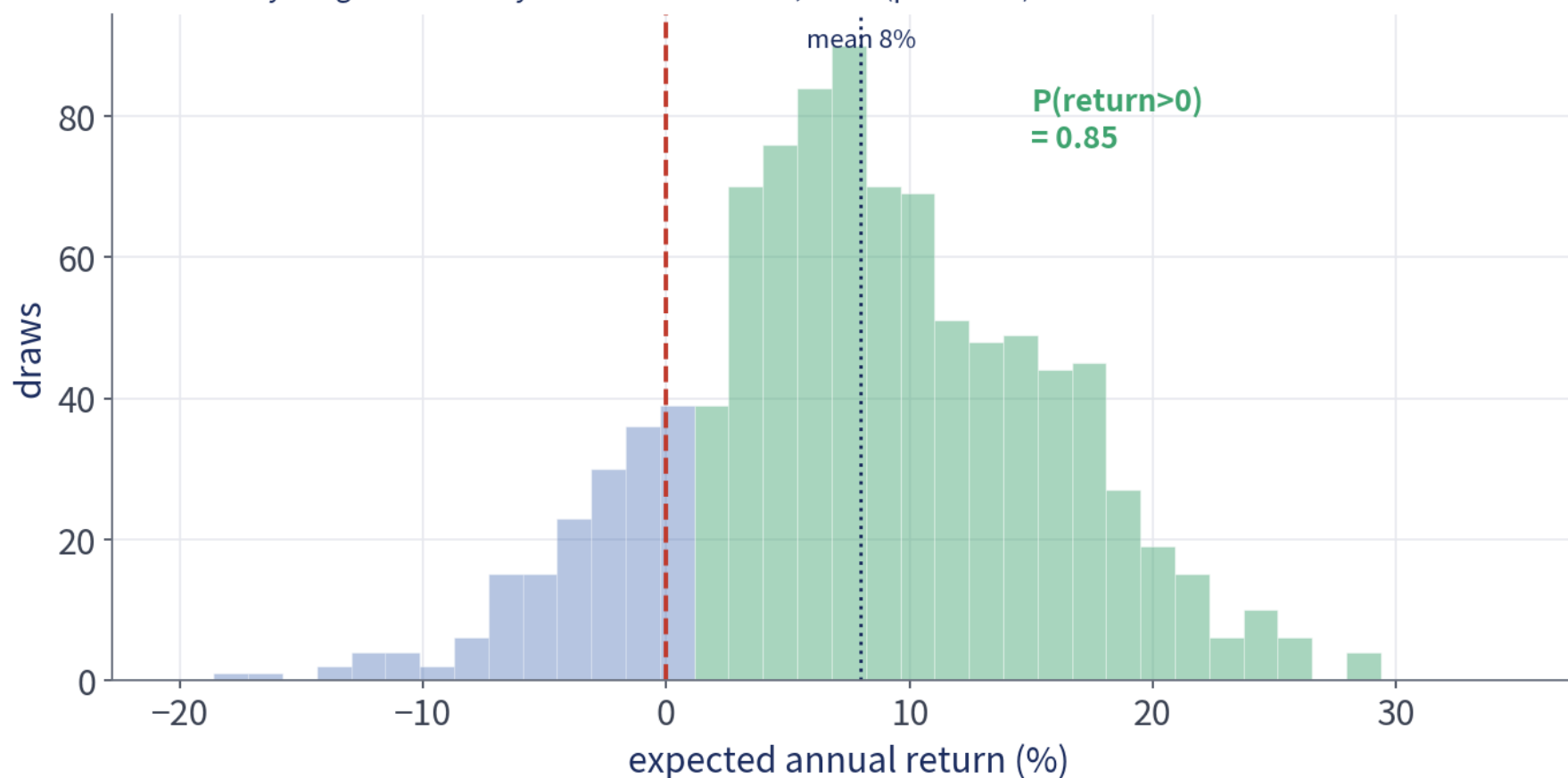
$$P(\text{return} > 0) = \frac{850}{1000} = \mathbf{0.85} \quad (\text{just count the winning draws})$$
$$\text{MC error} = \sqrt{\frac{0.85(0.15)}{1000}} = \mathbf{1.1\%}$$

Of 1000 draws, 850 are positive, so the probability the strategy is profitable is 0.85 - with a Monte Carlo error of about 1.1%. A probability is just a fraction of draws.

Everything Is a Summary of Draws

The histogram of posterior draws is the whole answer. Its balance point is the mean (8%); the green fraction to the right of zero is the probability of profit (0.85); and the spread tells you the uncertainty. One set of draws answers every question you can ask

Case A: everything is a summary of draws - the mean, and P(profitable)



Sample, Then Summarize

The whole plan in four words

What this buys you

- No integrals to solve, ever.
- Any quantity is a summary of the draws.
- Means, intervals, probabilities - all the same set.

Why it matters

- A profit probability is a decision input.
- It comes free from the draws you already have.
- The hard part is getting good draws.

How Many Draws?

Precision, and the cost of correlation

Monte Carlo answers come with an error that shrinks as 1 over the square root of the sample count - so quadrupling the draws only halves the error. We work out how many independent draws a target precision needs, then see why correlated MCMC draws, carrying less information each, demand far more

The setup

- Per-draw posterior spread of 4%.
- A target standard error of 0.1%.
- MCMC draws with 0.8 autocorrelation.

What we'll do by hand

- Apply the 1-over-root-N law.
- Solve for the samples needed.
- Adjust for the effective sample size.

The 1-Over-Root-N Law

Error and sample count

$$SE = \frac{\sigma}{\sqrt{N}} \Rightarrow \text{quadruple } N \text{ to halve the error}$$

The scaling

- The error is the spread over root-N.
- It falls slowly - only as a square root.
- Quadruple the draws to halve the error.

The implication

- High precision is expensive.
- Each extra digit costs a hundredfold.
- So efficient sampling matters.

Samples for a Target

Solve the law for N

$$N = \left(\frac{\sigma}{\text{SE}} \right)^2 = \left(\frac{4\%}{0.1\%} \right)^2 = 40^2 = \mathbf{1600} \text{ independent samples}$$

To pin the standard error to 0.1% with a 4% per-draw spread, you need 1600 independent draws - the ratio of spread to target, squared.

The Effective Sample Size

Correlated draws count for less

$$\text{ESS} = N \frac{1-\rho}{1+\rho} = N \frac{0.2}{1.8} = \frac{N}{9} \Rightarrow 1000 \text{ MCMC draws} \approx 111 \text{ independent}$$

The correction

- Autocorrelation shrinks the effective count.
- At 0.8, the factor is one-ninth.
- So 1000 draws act like about 111.

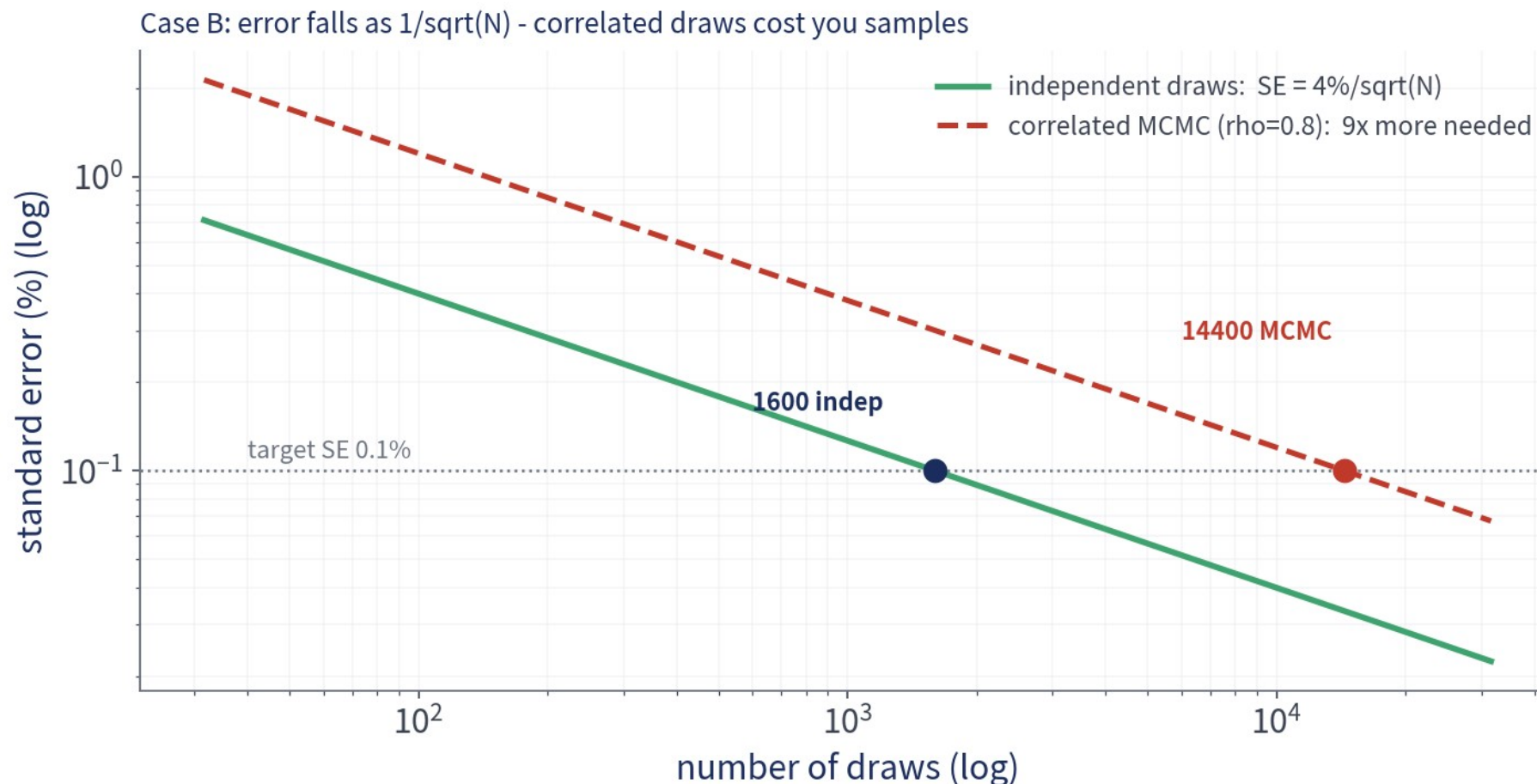
The cost

- You need nine times as many MCMC draws.
- That's 14,400 for the same precision.
- Correlation is the tax MCMC pays.

Error Falls Slowly, Correlation Costs

On a log-log plot the standard error is a straight line falling as $1/\sqrt{N}$. Independent draws hit the 0.1% target at 1600; correlated MCMC draws, each worth a ninth, need 14,400 to reach the same line.

The gap is the price of dependence



Correlation Is a Tax

Valid, but expensive

Why it matters

- MCMC draws are dependent by construction.
- Each carries less new information.
- So the effective sample size is smaller.

What to do

- Report effective sample size, not raw count.
- Reduce autocorrelation with better samplers.
- That's exactly what HMC and NUTS aim for.

Sample, Don't Integrate

The whole strategy in one line

Both cases rest on one idea: never solve the integral, sample it.

Draws give you every answer as a summary - a mean, a probability, an interval. The only question is how many draws you need, which the 1-over-root-N law answers - inflated by any correlation among them. Get good draws and the rest is arithmetic.

Estimation (Case A)

- The mean is an average of draws.
- A probability is a fraction of draws.
- The error is the spread over root-N.

Convergence (Case B)

- Error falls as 1 over root-N.
- A target precision fixes the count.
- Correlation inflates what you need.

Reproduce the Estimates and the Counts

Direct Claude Code to check your arithmetic

GOAL Verify the Monte Carlo estimates and the sample-count arithmetic, then simulate.

[INPUT] Prompt · Claude Code

Write and run `part1_practice.py` that:

CASE A - estimation:

- * average the draws [5,8,11,6,10] for the mean;
- * from 1000 draws with 850 positive, print $P(\text{profit})$ and its MC standard error.

CASE B - convergence:

- * with per-draw sd 4%, print SE for $N=100, 400, 1000$;
- * solve for N at $SE=0.1\%$;
- * with $\rho=0.8$, print the ESS factor and the MCMC draws needed.

Then draw samples and confirm the $1/\sqrt{N}$ law.

Check against the slides

- mean 8%, $P(\text{profit})$ 0.85.
- MC error $\sim 1.1\%$.
- $N = 1600$ independent.
- $ESS = N/9 \rightarrow 14,400$ MCMC.

Deliverable: `part1_practice.py` whose printed numbers match the estimates and the sample counts.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 How does an expectation become an average?

**Monte Carlo: average
f over the draws**

Q2 How does the error scale with sample count?

**As 1 over the square
root of N**

Q3 Why do correlated draws cost more?

**The effective sample
size is smaller**

All three come from Monte Carlo estimation and the 1-over-root-N law.

Three Pitfalls to Avoid

Where sampling trips people

Common mistakes

- "More draws always fix the error fast."
- "Report a Monte Carlo number with no error."
- "Count MCMC draws as if independent."

The reality

- Error falls only as root-N - it's slow.
- Always attach the Monte Carlo standard error.
- Use effective sample size for correlated draws.

Two Calculations to Remember

Part 1 applications in one screen

1

Every answer is a summary of draws

The posterior mean is the average of the draws (8%), the probability of profit is the fraction above zero (0.85), and its Monte Carlo error is about 1.1%. No integral is ever solved - the samples carry everything.

2

Precision costs samples, correlation costs more

Error falls only as $1/\sqrt{N}$, so quadrupling draws halves the error; pinning it to 0.1% needs 1600 independent draws. With 0.8 autocorrelation the effective sample size is a ninth, so MCMC needs 14,400.

3

Sample, don't integrate

The strategy is to replace an intractable integral with an average over draws. The only real question is how to generate good, low-correlation draws - which is what the rest of the week builds.

Next — Part 2 practice: the Metropolis and Gibbs samplers that actually generate the draws.

★ WEEK 14 · PART 2 of 3

Metropolis-Hastings and Gibbs

The Algorithms That Build the Chain

Propose and accept · the evidence cancels · tuning · Gibbs sampling

Part 1 said: build a chain whose stationary law is the posterior. Here is how. Metropolis-Hastings proposes moves and accepts them by a simple ratio; Gibbs draws one parameter at a time.

2

PART 2 · THE ALGORITHMS

Building the Chain

The Metropolis-Hastings accept-reject rule, why the intractable evidence cancels, tuning the proposal step, and Gibbs sampling as the conditional-by-conditional alternative.

From Idea to Algorithm

Part 1 gave the plan; Part 2 builds the chain

1

We want posterior samples

Part 1: build a Markov chain whose stationary distribution is the posterior, then read answers off the draws.

2

But how do we build such a chain?

We need a concrete rule for choosing the next state that guarantees the right long-run distribution.

3

Two classic recipes do it

Metropolis-Hastings proposes and accepts moves; Gibbs updates one parameter at a time. Both give a valid chain.

Part 2 in one line: propose-and-accept (Metropolis-Hastings) or draw-conditionals (Gibbs) build the chain.

The Metropolis-Hastings Algorithm

Propose a move, then decide whether to take it

Metropolis-Hastings is disarmingly simple: from the current point, propose a nearby candidate. If it makes the target higher, take it; if lower, take it only sometimes. Repeat, and the chain traces out the posterior.

The loop

- Propose a candidate near the current point.
- Compare the target at the candidate and the current point.
- Accept or reject by a simple probability rule.

Why it is remarkable

- It needs only the target's value at two points.
- No integral, no normalizer, no special structure.
- It works for an enormous range of models.

The Acceptance Rule

One ratio decides every move

$$\text{acceptance: } \alpha = \min\left(1, \frac{p(\theta^*|\text{data})}{p(\theta|\text{data})}\right) \text{ accept } \theta^* \text{ with prob. } \alpha$$

Reading it

- Form the ratio of the target at candidate over current.
- If the candidate is better, the ratio exceeds one — always accept.
- If worse, accept it with probability equal to the ratio.

The effect

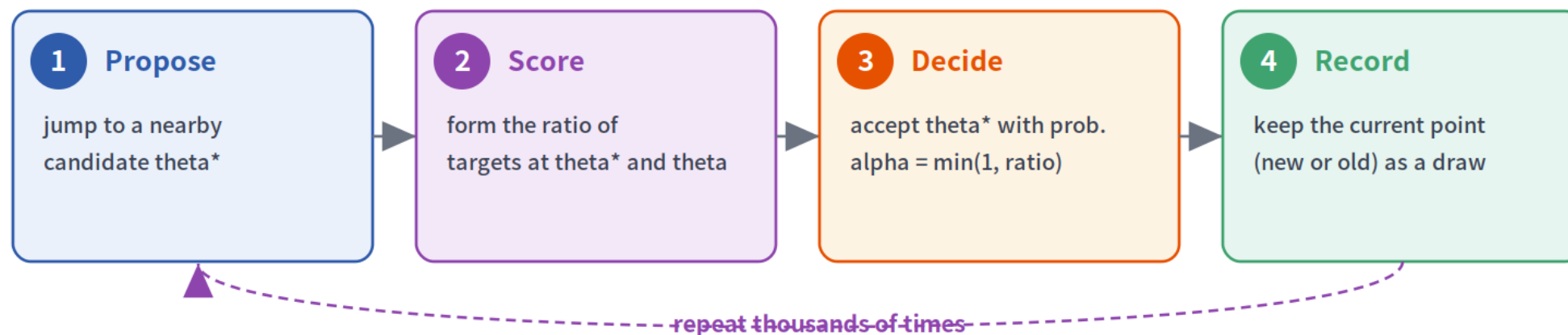
- The chain climbs toward high-probability regions.
- But it still wanders into lower ones sometimes.
- That balance is what maps the whole distribution.

One Metropolis-Hastings Step

Propose a candidate, score it by the ratio of targets, accept with the resulting probability, and record the point. Repeat thousands of times, and the recorded draws reproduce the posterior.

One Metropolis-Hastings Step

Propose a move, score it by a ratio, then accept or reject - and repeat.



Why it works

Uphill moves are always accepted; downhill moves sometimes are. That balance makes the chain linger where the target is high - so its long-run visits reproduce the posterior. The normalizer never appears.

The Trick: The Evidence Cancels

Why we never need the hard integral

Here is the magic. The acceptance rule uses a RATIO of posteriors. The intractable normalizer appears in both the top and the bottom — so it cancels. We only ever need the likelihood times the prior.

What cancels

- The posterior is likelihood times prior, over the evidence.
- In the ratio, the evidence is in both numerator and denominator.
- So it divides out completely.

Why this is everything

- The evidence was the whole obstacle (Week 13).
- MCMC never has to compute it.
- That is why the method works where exact Bayes fails.

The Evidence Divides Out

Seeing it happen

$$\frac{p(\theta^*|\text{data})}{p(\theta|\text{data})} = \frac{P(\text{data}|\theta^*)P(\theta^*)/P(\text{data})}{P(\text{data}|\theta)P(\theta)/P(\text{data})} \Rightarrow P(\text{data}) \text{ cancels}$$

The two posteriors

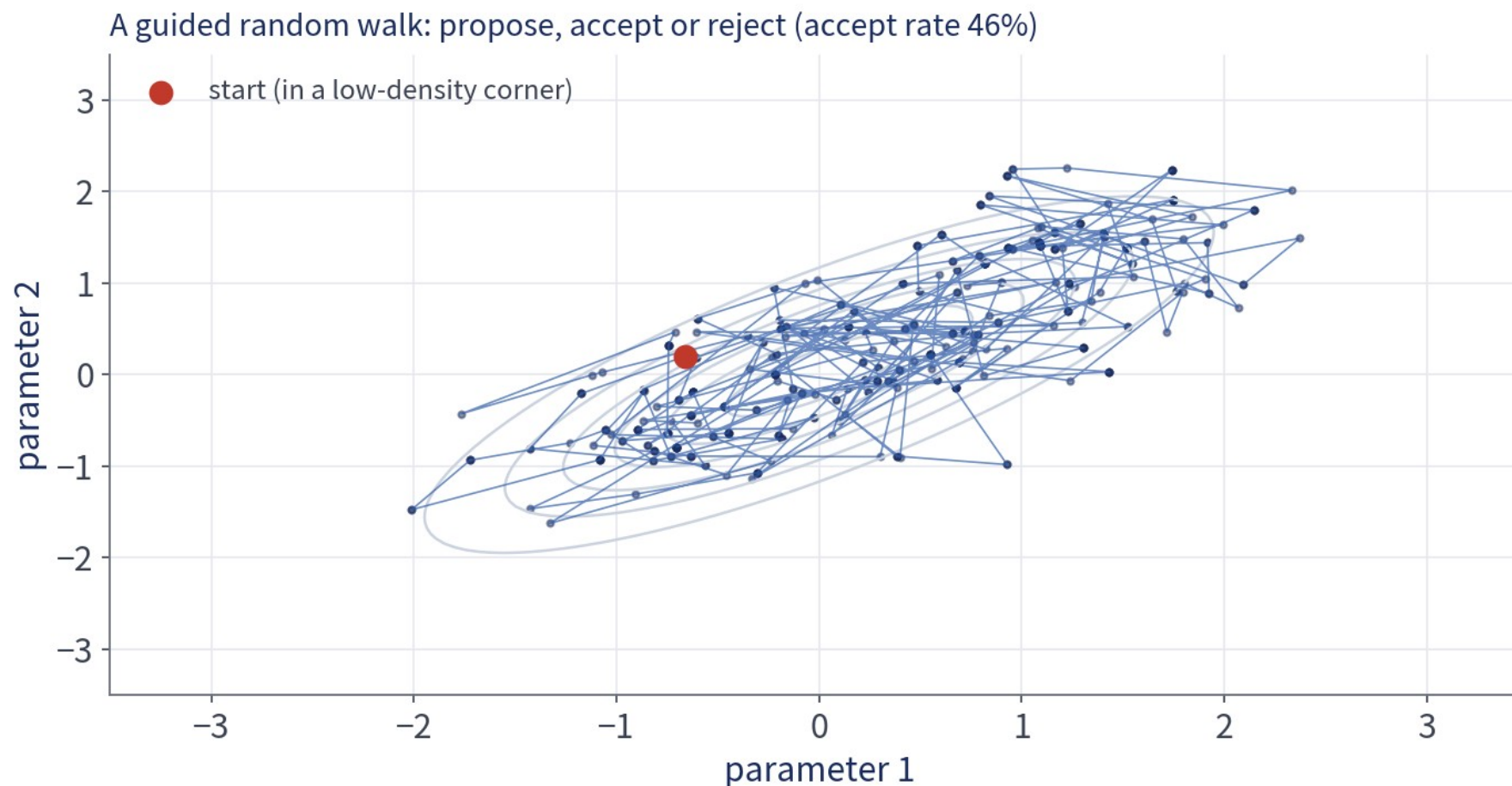
- Both are likelihood times prior, over the same evidence.
- That evidence is the identical constant in each.
- It does not depend on the parameter.

The result

- The constant cancels in the ratio.
- Only likelihood times prior remains — computable.
- The hard integral simply never appears.

A Guided Random Walk

Starting in a low-density corner, the chain proposes moves and accepts or rejects them, quickly finding and then exploring the high-probability region traced by the target's contours.



Why Accept Some Worse Moves?

Exploration versus exploitation

If we only ever climbed

- The chain would rush to the peak and stop.
- It would sample one point, not the distribution.
- We would learn the mode, not the uncertainty.

By sometimes stepping down

- The chain explores the full spread of the posterior.
- It visits the tails in the right proportion.
- That is what makes the draws represent the whole shape.

The Proposal Distribution

How candidates are generated

What it is

- The rule for proposing a candidate from the current point.
- Often a Gaussian step of some chosen width.
- It is the one thing you must design.

Why it matters

- It sets how far the chain tries to move each step.
- Too timid or too bold both hurt (next slide).
- Getting it right is most of the practical art.

Tuning the Proposal Step

Too small, too big, or just right

The proposal width is a dial you must tune. Too small and the chain inches along, exploring painfully slowly. Too big and almost every move is rejected, so it gets stuck. The sweet spot moves freely AND accepts often enough.

The two failure modes

- Too small: high acceptance, but tiny steps — slow.
- Too big: bold jumps, but constant rejection — stuck.
- Both explore the posterior badly.

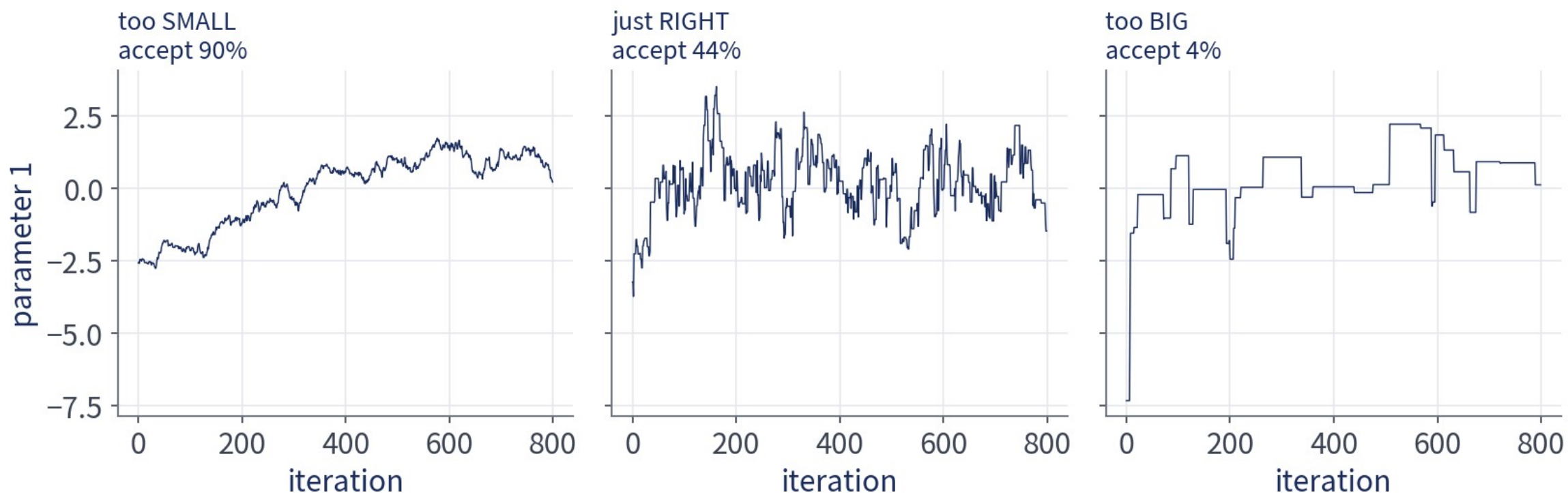
The sweet spot

- Steps large enough to move across the distribution.
- Small enough that many are accepted.
- A moderate acceptance rate is the sign of good tuning.

Too Small, Just Right, Too Big

Three chains, differing only in proposal width. A tiny step crawls and barely moves; a huge step gets stuck rejecting; a well-chosen step mixes freely across the distribution.

Tuning the proposal step: too small crawls, too big gets stuck, just right mixes



The Target Acceptance Rate

A dial you can read

What to watch

- The fraction of proposals the chain accepts.
- Very high means the steps are too timid.
- Very low means the steps are too bold.

The guidance

- For random-walk moves, aim for a moderate rate.
- Roughly a quarter to a half is a common target.
- Modern samplers tune this automatically.

Gibbs Sampling

Update one parameter at a time

Gibbs sampling takes a different route. Instead of proposing joint moves, it cycles through the parameters, drawing each from its distribution given all the others. No proposal, no rejection — every draw is accepted.

The idea

- Fix all parameters but one.
- Draw that one from its full conditional distribution.
- Cycle through them all, then repeat.

The appeal

- No proposal width to tune.
- No rejected moves — every step advances.
- Clean and efficient when the conditionals are known.

The Gibbs Update

Draw each conditional in turn

Gibbs: draw each in turn $\theta_j \sim p(\theta_j \mid \theta_{-j}, \text{data})$

Reading it

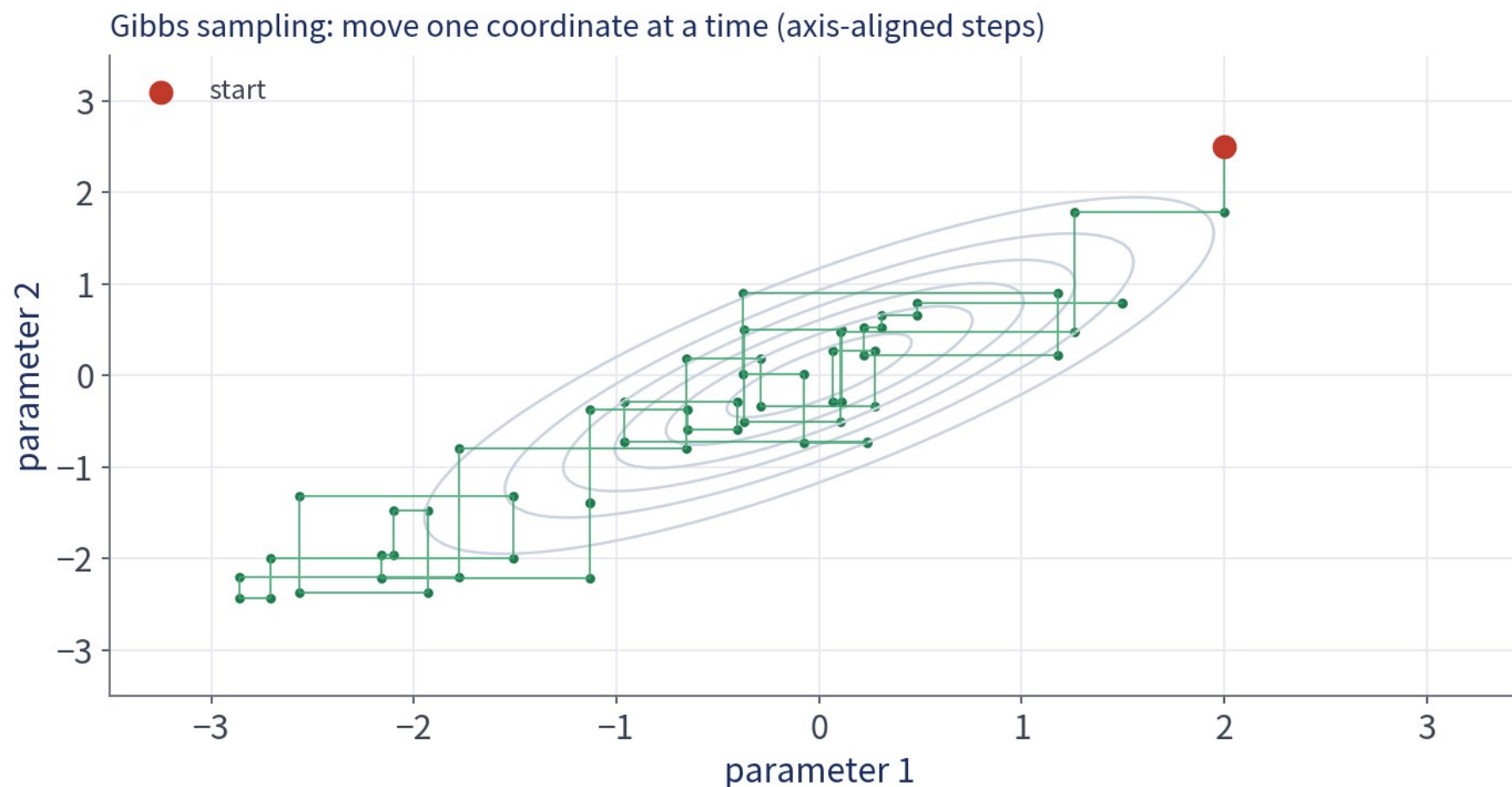
- For each parameter, condition on all the others.
- Draw it from that full conditional distribution.
- Sweep through every parameter, then loop.

The requirement

- You need each full conditional in closed form.
- Conjugate models often provide exactly this.
- When you have them, Gibbs is very clean.

Gibbs Moves One Coordinate at a Time

Each step is axis-aligned: hold one parameter, draw the other from its conditional. The zig-zag path fills the same target, but its stair-step motion can be slow when parameters are correlated.



Two Ways to Build the Same Chain

Metropolis-Hastings proposes and rejects and works almost anywhere, but must be tuned. Gibbs draws exact conditionals with no tuning or rejection, but needs those conditionals and struggles with correlation

Two Ways to Build the Same Chain

Metropolis-Hastings proposes and rejects; Gibbs draws exact conditionals, one at a time.

Metropolis-Hastings

- Propose a move, accept or reject.
- Needs only the target, up to a constant.
- Works for almost any model.
- Must tune the proposal step.
- Rejections waste some proposals.

General-purpose workhorse.

Gibbs sampling

- Update one parameter at a time.
- Draw each from its full conditional.
- No tuning, no rejections.
- Needs those conditionals in closed form.
- Struggles when parameters are correlated.

Elegant when conditionals are known.

When to Use Which

Matching sampler to model

Reach for Metropolis-Hastings

- The model is arbitrary or non-conjugate.
- You have no closed-form conditionals.
- You want one general-purpose tool.

Reach for Gibbs

- The full conditionals are known and easy to sample.
- The model is conjugate in its pieces.
- You want to avoid tuning and rejections.

Why the Chain Has the Right Target

The balance condition, in words

The guarantee

- The accept-reject rule is built to satisfy a balance condition.
- It makes flows between states cancel at the posterior.
- That forces the posterior to be the stationary distribution.

What it means for you

- You don't have to prove this each time.
- The standard rules come with the guarantee.
- Run the chain, and it targets the posterior by design.

Both Build the Same Kind of Chain

Different moves, same destination

What they share

- Both produce a Markov chain over the parameters.
- Both have the posterior as their stationary law.
- Both yield correlated-but-valid posterior draws.

How they differ

- MH proposes joint moves and may reject.
- Gibbs updates coordinates and always accepts.
- Gibbs is in fact a special case of the MH family.

What Part 3 Adds

From a running chain to trustworthy answers

Diagnosing the chain

- Trace plots: is it mixing well?
- Burn-in: discard the awkward start.
- Convergence checks before you trust the draws.

Doing it for real

- Modern tools like PyMC that sample and tune for you.
- Fitting the messy models from earlier weeks.
- The bridge to Bayesian state-space models.

Code a Metropolis Sampler by Hand

Direct Claude Code to demystify the algorithm

GOAL Build a Metropolis sampler from scratch and watch it recover a target distribution.

[INPUT] Prompt · Claude Code

Write and run metropolis.py that:

1. defines a target as an UNNORMALIZED density (e.g. a mixture of two Gaussians);
 2. runs a random-walk Metropolis sampler using only the accept ratio $\min(1, \text{target}^*/\text{target})$;
 3. tries three proposal widths (too small, good, too big) and reports each acceptance rate;
 4. overlays the sampled histogram on the target.
- Confirm the histogram matches the target and note which proposal width mixes best.

How to work with it

- Only the target ratio is needed.
- The normalizer never appears.
- Tune the width to a moderate accept rate.
- The histogram should match the target.

Deliverable: metropolis.py + sampled histograms at three step sizes with acceptance rates.

Three Questions Before You Move On

If you can answer these, Part 2 has landed

Q1 In the MH acceptance rule, what makes the normalizer cancel?

It's a ratio of posteriors

Q2 Why does MH sometimes accept a worse move?

To explore the whole posterior

Q3 What does Gibbs sampling draw each parameter from?

Its full conditional

All three come from the acceptance ratio, exploration, and the Gibbs conditional draws.

Three Misconceptions to Avoid

Where sampler intuition slips

Common myths

- "A higher acceptance rate is always better."
- "Gibbs is a totally different method from MH."
- "Rejected proposals are wasted and skipped."

The reality

- Too-high acceptance means tiny, slow steps — tune it down.
- Gibbs is a special case of the MH family.
- On rejection you re-record the CURRENT point as a draw.

Three Things to Carry Forward

Part 2 in one screen

1

Metropolis-Hastings proposes and accepts

From the current point, propose a candidate and accept it by a simple ratio of targets. Uphill moves are always taken; downhill moves sometimes are, so the chain maps the whole posterior.

2

The evidence cancels

Because the acceptance rule is a ratio of posteriors, the intractable normalizer divides out. MCMC needs only the likelihood times the prior — which is why it works where exact Bayes fails.

3

Gibbs is the conditional-by-conditional alternative

It draws each parameter from its full conditional in turn — no tuning, no rejection — when those conditionals are available. Both methods build a chain with the posterior as its stationary law.

Next — Part 3: trace plots, burn-in, convergence diagnostics, and MCMC in practice with PyMC.

★ WEEK 14 · PART 2 · PRACTICE

Metropolis and Gibbs, Worked by Hand

Take One Step, Tune the Proposal, or Skip the Rejection

The acceptance ratio and the sampler's one dial

Part 2's samplers, with the arithmetic shown. We take one Metropolis-Hastings step - watching the evidence cancel and a worse move get accepted - then tune the proposal by its acceptance rate, and see how Gibbs skips rejection entirely.

2P

PART 2 · APPLICATIONS

Propose, Accept, or Tune

Case A: one Metropolis-Hastings step - the acceptance ratio, the evidence cancelling, and why a worse move is sometimes taken. Case B: the acceptance-rate dial that tunes the proposal, and Gibbs as the no-rejection alternative. Then a Claude Code lab.

What Part 2 Told Us

Now we take a step - with numbers

1

Propose a move, then decide

Metropolis-Hastings proposes a candidate and accepts it with a probability set by one ratio of target densities.

2

The evidence cancels

Because the ratio divides two posteriors, the intractable normalizer drops out - you never compute the hard integral.

3

Tune the proposal, or use Gibbs

The proposal step size sets the acceptance rate, which you tune. Gibbs instead draws each conditional exactly - no rejection.

Everything here is a calculator exercise: one acceptance ratio, three step sizes, one conditional.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from a current state or a step size.
- Compute the ratio, or the acceptance rate.
- Then read the accept, reject, or tune.

What you should watch

- How the evidence cancels in the ratio.
- How a worse move can still be accepted.
- How the step size sets efficiency.

One Metropolis-Hastings Step

Propose, score, accept or reject

Sampling the posterior of a strategy's expected return, we sit at 8% and propose two moves. Each is scored by the ratio of unnormalized posteriors - likelihood times prior - so the intractable evidence cancels. An uphill move is always taken; a downhill one is accepted with probability equal to the ratio. We walk both by hand

The current state

- At $\theta = 8\%$.
- Unnormalized target: $0.50 \times 0.60 = 0.30$.
- Likelihood times prior.

What we'll do by hand

- Score an uphill proposal to 11%.
- Score a downhill proposal to 4%.
- Decide each with a uniform draw.

The Acceptance Rule

One ratio, and the evidence cancels

$$a = \min\left(1, \frac{p(\theta'|D)}{p(\theta|D)}\right) = \min\left(1, \frac{L(\theta')\pi(\theta')/P(D)}{L(\theta)\pi(\theta)/P(D)}\right) \quad (\text{the evidence cancels})$$

The ratio

- Divide the proposed target by the current.
- Both are likelihood times prior.
- Accept with that ratio, capped at one.

Why it's magic

- The evidence sits in both, top and bottom.
- So it cancels - never computed.
- That's what makes MCMC possible.

An Uphill Move

Higher target, always accepted

$$\text{uphill } \theta=8\% \rightarrow \theta'=11\% : \text{ ratio} = \frac{0.75(0.56)}{0.50(0.60)} = \frac{0.42}{0.30} = \mathbf{1.40} > 1 \Rightarrow \text{ACCEPT}$$

The calculation

- Proposed target: 0.75 times 0.56 = 0.42.
- Divide by the current 0.30.
- The ratio is 1.40.

The decision

- A ratio above one means always accept.
- The move improves the target.
- The chain steps to 11%.

A Downhill Move

Sometimes taken anyway

$$\text{downhill } \theta''=4\% : \text{ratio} = \frac{0.18}{0.30} = \mathbf{0.60}$$

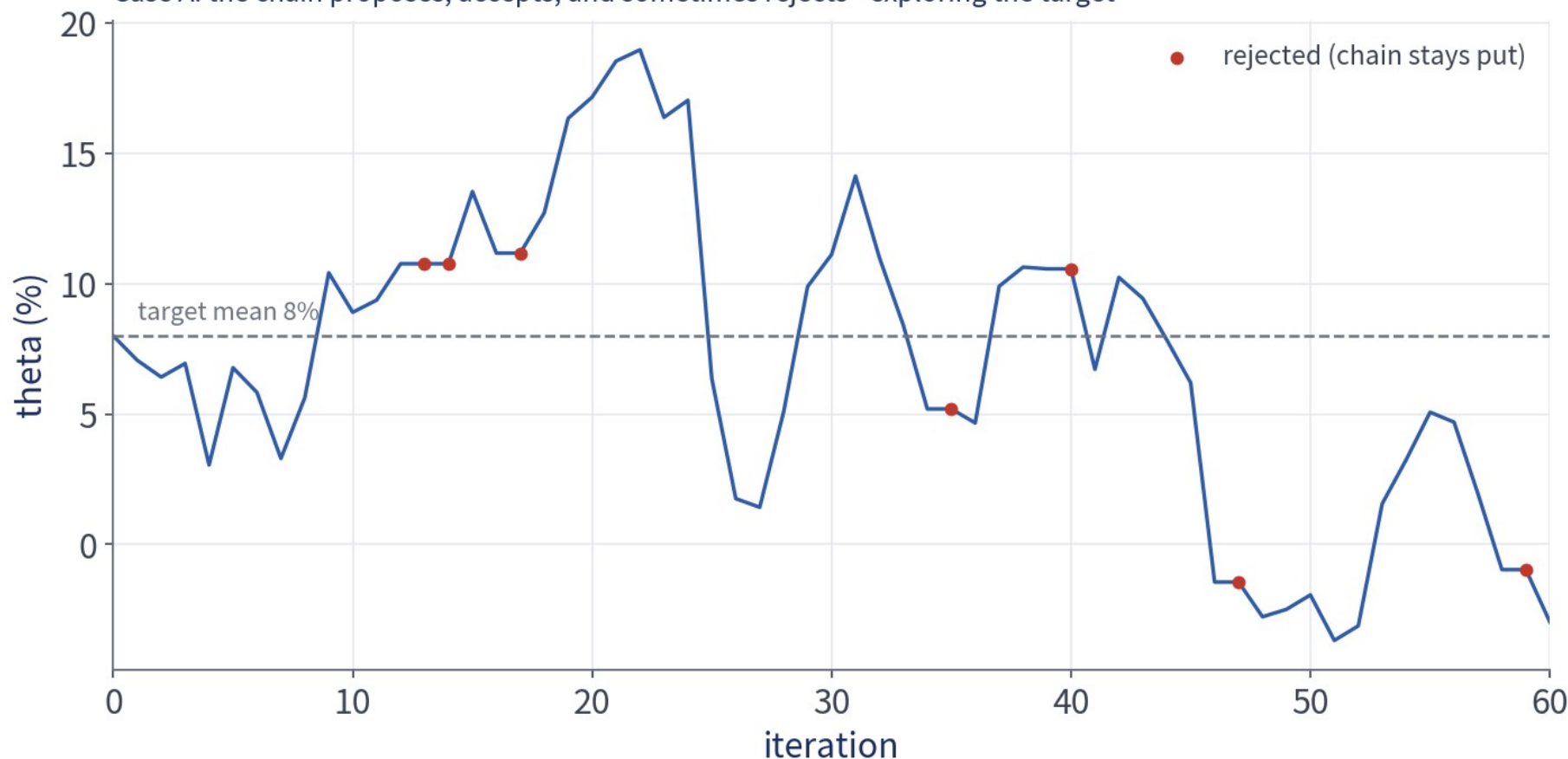
draw $u = 0.45 < 0.60 \Rightarrow$ ACCEPT (a worse move, for exploration)

The downhill move to 4% scores a ratio of 0.60, so it's accepted 60% of the time. A uniform draw of 0.45 falls below 0.60, so the chain takes the worse move - that's exploration.

Proposing, Accepting, Rejecting

The chain wanders around the target's center of 8%, taking most uphill moves and some downhill ones. The red points mark rejected proposals, where it stays put. Over many steps this exploration traces out the whole posterior - the accepted states are the samples

Case A: the chain proposes, accepts, and sometimes rejects - exploring the target



Why Accept Worse Moves

Exploration beats greed

If it only climbed

- The chain would rush to the peak and stop.
- It would never see the tails.
- The samples would misrepresent the posterior.

By taking some losses

- It explores the whole distribution.
- It visits regions in proportion to their mass.
- That's what makes the samples valid.

Tune the Proposal, or Skip It

The acceptance dial and Gibbs

Metropolis-Hastings has one real knob: the proposal step size, which sets the acceptance rate. Too small and it crawls, accepting almost everything but barely moving; too large and it stalls, rejecting almost everything. The sweet spot is in between. Gibbs avoids the whole issue by drawing each conditional exactly

The dial

- Small steps: acceptance near 0.90, tiny moves.
- Large steps: acceptance near 0.06, mostly stuck.
- The target is roughly a third.

What we'll do by hand

- Read the acceptance rate at three step sizes.
- Identify the efficient one.
- Contrast with a Gibbs update.

The Acceptance-Rate Dial

Efficiency lives in the middle

step too small $\Rightarrow$ accept ≈ 0.90 (crawls); too large $\Rightarrow 0.06$ (stuck); sweet spot $\approx 0.23\text{--}0.44$

Both extremes fail

- High acceptance: steps too small to explore.
- Low acceptance: chain rejects and sticks.
- Either way, the samples are correlated.

The rule of thumb

- Aim near 0.44 in one dimension,
- and near 0.234 in many dimensions.
- That maximizes effective samples.

Three Step Sizes

Read the acceptance rate

step size	acceptance	behavior
small (0.5)	0.90	tiny steps, high autocorrelation
good (2.5)	0.35	near the 0.23-0.44 sweet spot
large (8.0)	0.06	mostly rejected, chain stuck

The small step accepts almost everything but crawls; the large one rejects almost everything and sticks. Only the middle step - about a third acceptance - mixes efficiently.

The Gibbs Alternative

Draw each conditional, no rejection

Gibbs: draw each conditional in turn $\theta_1 \sim p(\theta_1 \mid \theta_2, D)$, $\theta_2 \sim p(\theta_2 \mid \theta_1, D)$ (no rejection)

How it works

- Update one parameter at a time.
- Draw it from its conditional, holding others fixed.
- Every draw is accepted - no tuning.

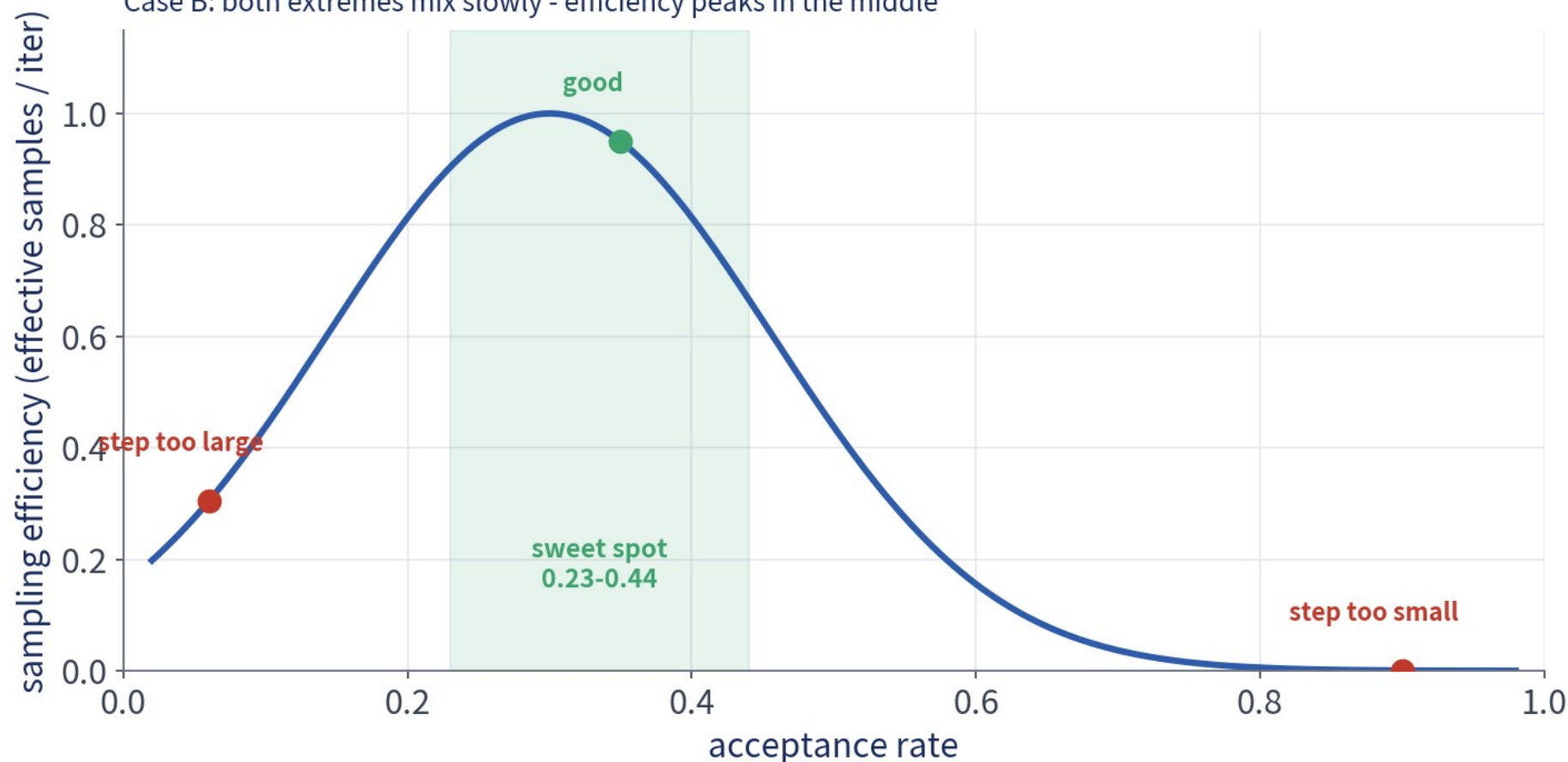
The catch

- You need the conditionals in closed form.
- Great for conjugate models,
- harder when they don't factor nicely.

Efficiency Peaks in the Middle

Sampling efficiency - effective samples per iteration - is low at both extremes and peaks in the shaded 0.23-to-0.44 band. A 0.90 acceptance means tiny useless steps; a 0.06 acceptance means a stuck chain. The good step size sits right at the top of the hump

Case B: both extremes mix slowly - efficiency peaks in the middle



Two Ways to the Same Chain

Tune, or draw exactly

Metropolis-Hastings

- Works almost anywhere - just need the target.
- But you must tune the proposal.
- The acceptance rate is your guide.

Gibbs

- No tuning, no rejection.
- But needs closed-form conditionals.
- Ideal for conjugate structure.

Build the Chain Two Ways

Propose-and-reject chain - ~~Both samplers construct a chain whose long-run law is the posterior. Metropolis-Hastings proposes and rejects, using only the target ratio - so the evidence cancels - but needs its step size tuned to an efficient acceptance rate. Gibbs sidesteps rejection by drawing each conditional exactly, when you can. Same destination, two~~

The step (Case A)

- One ratio decides each move.
- The evidence cancels out.
- Worse moves are taken for exploration.

The tuning (Case B)

- Acceptance rate sets efficiency.
- Aim for the 0.23-0.44 band.
- Or use Gibbs and skip it.

Reproduce the Step and the Tuning

Direct Claude Code to check your arithmetic

GOAL Verify one MH step, the acceptance-rate tuning, and a Gibbs update.

[INPUT] Prompt · Claude Code

```
Write and run part2_practice.py that:  
CASE A - one MH step:  
  * current target 0.30; score proposals 0.42 and  
    0.18; print each ratio and the accept decision  
    (use  $u=0.45$  for the downhill move).  
CASE B - tuning:  
  * run MH on a target at step sizes 0.5, 2.5, 8.0;  
    print the empirical acceptance rate for each;  
  * also code a 2-parameter Gibbs sampler.  
Then plot the trace and the efficiency vs acceptance.  
Match every printed number to the slides.
```

Check against the slides

- uphill ratio 1.40 -> accept.
- downhill 0.60, $u=0.45$ -> accept.
- accept rates 0.90, 0.35, 0.06.
- Gibbs: acceptance = 1.

Deliverable: part2_practice.py whose printed numbers match the MH step and the acceptance rates.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 Why does the evidence cancel in the ratio?

It appears in both posteriors, top and bottom

Q2 Why accept some worse moves?

To explore the whole posterior, not just the peak

Q3 When would you use Gibbs over Metropolis?

When the conditionals are available in closed form

All three come from the MH acceptance step and the tuning-versus-Gibbs choice.

Three Pitfalls to Avoid

Where the samplers trip people

Common mistakes

- "A high acceptance rate means good sampling."
- "Compute the evidence to score a move."
- "Gibbs always beats Metropolis."

The reality

- Too high means tiny steps - it barely explores.
- The evidence cancels; only the ratio matters.
- Gibbs needs closed-form conditionals to work.

Two Calculations to Remember

Part 2 applications in one screen

1

One MH step is a ratio and a coin flip

From a target of 0.30, the uphill move to 11% scored 1.40 and was accepted; the downhill move to 4% scored 0.60 and was accepted anyway when a uniform draw of 0.45 fell below it. The evidence cancelled in both ratios.

2

Tune the proposal by its acceptance rate

A tiny step accepted 90% but crawled; a large one accepted 6% and stuck; the good step accepted about a third - near the 0.23-0.44 sweet spot where efficiency peaks. Gibbs skips this by drawing each conditional exactly.

3

Two routes, one chain

Metropolis-Hastings proposes and rejects using only the target ratio, tuned to a good acceptance rate; Gibbs draws conditionals with no rejection. Both build a chain whose long-run law is the posterior.

Next — Part 3 practice: Hamiltonian Monte Carlo and NUTS, which propose smarter, farther moves.

★ WEEK 14 · PART 3 of 3

Hamiltonian Monte Carlo and NUTS

Roll a Ball, Don't Wander

The physical picture · leapfrog · the U-turn · the modern default

A random walk explores blindly. HMC treats the parameter as a ball rolling on a landscape shaped by the posterior — one kick carries it far in a smart direction. NUTS decides how far to roll, automatically.

3

PART 3 · HMC & NUTS

The Sampler That Rolls

Why random walks are slow, the physics of Hamiltonian Monte Carlo, leapfrog integration, the U-turn problem, and how NUTS turns HMC into the default engine behind modern tools.

We Can Sample — But Slowly

Parts 1-2 built the chain; can we do better?

1

Metropolis-Hastings works anywhere

Propose a nearby move, accept by a ratio. Simple and general — but it wanders like a blindfolded walker.

2

Random walks are slow in high dimensions

With many parameters, tiny aimless steps take forever to explore the posterior. Draws stay highly correlated.

3

HMC uses physics to move smartly

Treat the parameter as a rolling ball guided by the posterior's shape. One move travels far — and NUTS tunes how far.

Part 3 in one line: HMC rolls a ball across the posterior; NUTS decides when to stop rolling.

The Trouble With Random Walks

Aimless steps waste effort

A random-walk sampler proposes blindly — a small step in a random direction, with no sense of where the posterior is. In high dimensions almost every blind direction is wrong, so it inches along and its draws stay heavily correlated.

Why it struggles

- Each proposal ignores the shape of the target.
- Big steps get rejected; small steps barely move.
- In many dimensions, random directions rarely help.

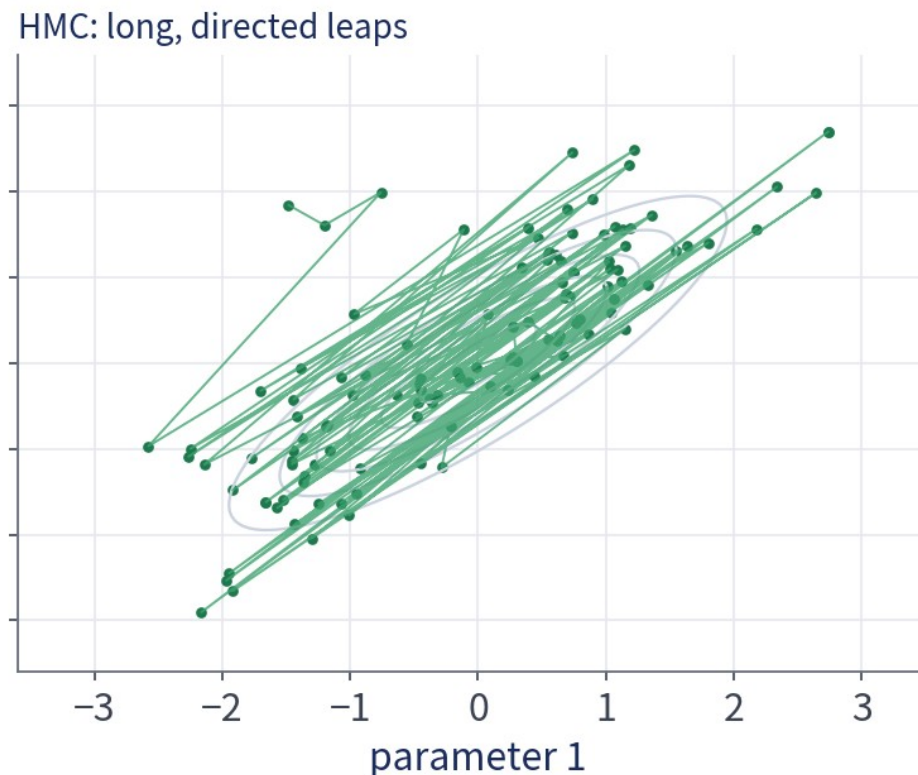
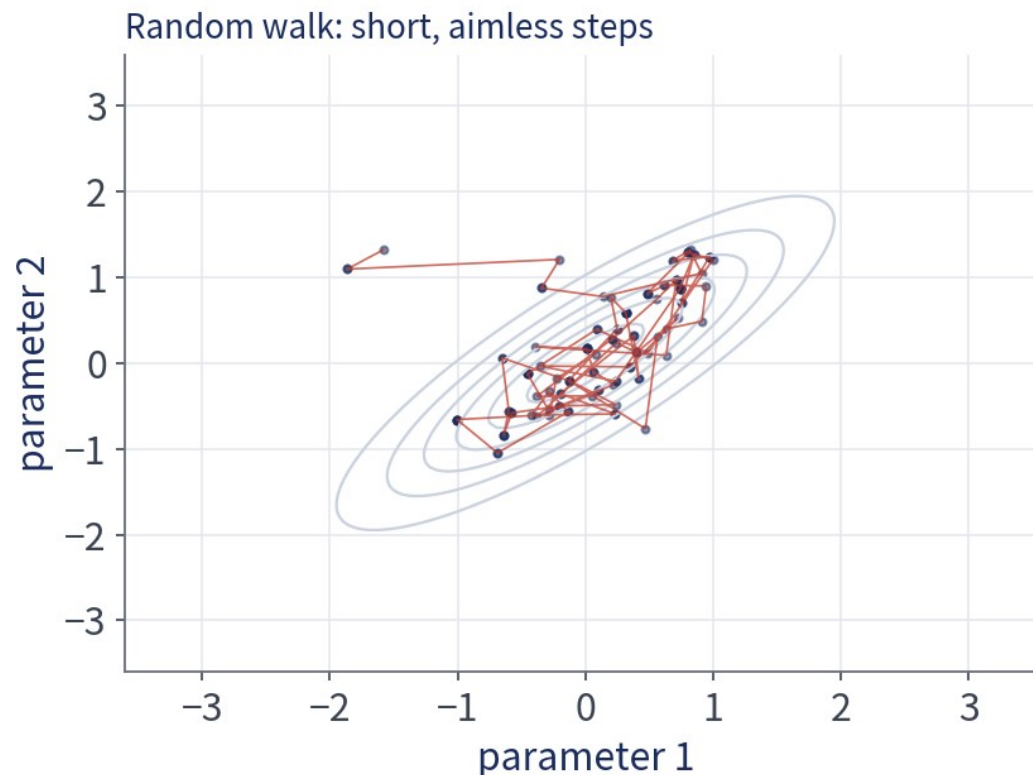
The cost

- The chain explores painfully slowly.
- Consecutive draws are highly correlated.
- You need enormous chains for decent precision.

Random Walk vs HMC

The same number of steps on the same target. The random walk (left) shuffles around in short, aimless hops; HMC (right) makes long, directed leaps that cover the whole distribution quickly.

Same number of steps: HMC covers the distribution far faster



Roll a Ball, Don't Wander

Borrow the physics of motion

HMC's insight is physical. Picture the posterior flipped into a valley — high posterior is low ground. Place a ball, give it a random kick, and let it roll. Gravity pulls it toward the low ground, so it naturally visits high-posterior regions.

The mental model

- The parameter is a ball on a landscape.
- The landscape is the posterior, turned upside down.
- A random kick sets it rolling; you follow the path.

Why it is smart

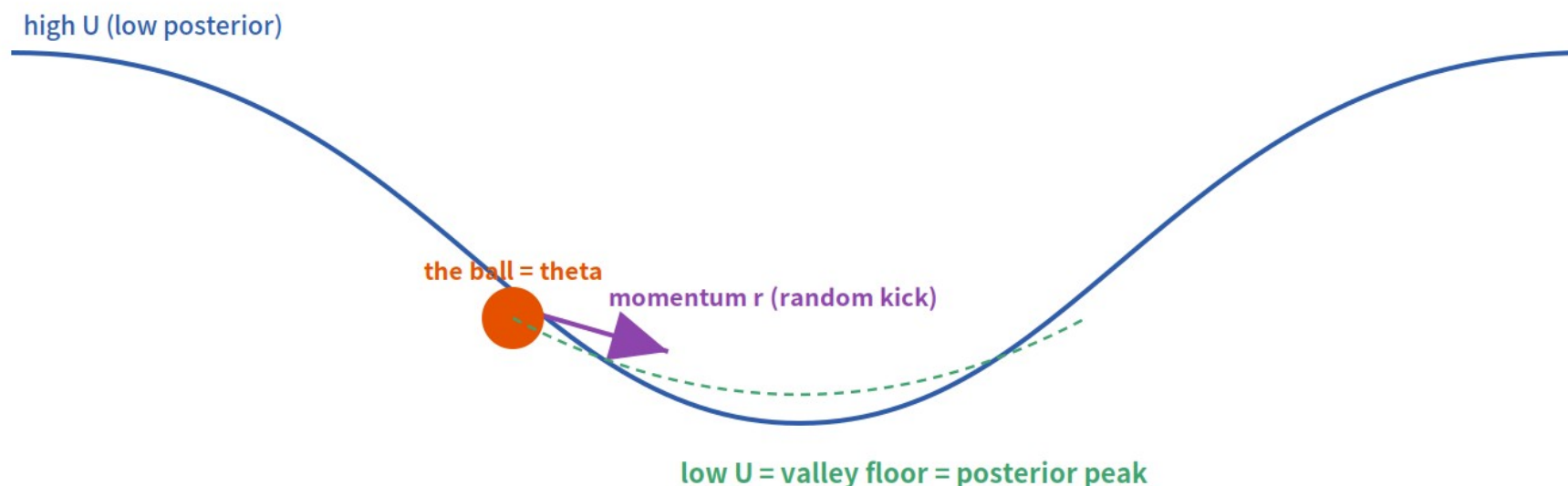
- The ball uses the slope to move in useful directions.
- One roll carries it far across the distribution.
- No blind guessing — the geometry guides it.

The Physical Picture

Flip the posterior into a valley: its peak becomes the valley floor. The ball is the parameter, the arrow is a random momentum, and the slope of the landscape pulls the ball toward high-posterior ground.

The Physical Picture

Turn the posterior upside-down into a valley, then roll a ball across it.



The ball is pulled downhill by the SLOPE of the landscape (the gradient) - so it naturally heads toward high-posterior regions.

Potential Energy = Negative Log Posterior

The shape the ball rolls on

$$\underbrace{U(\theta)}_{\text{potential energy}} = -\log p(\theta \mid \text{data}) \quad (\text{the landscape the ball rolls on})$$

What it means

- Take the log of the posterior and flip its sign.
- High posterior becomes low 'ground' — a valley.
- This surface is the potential energy.

The intuition

- The ball is pulled toward low potential.
- Low potential means high posterior.
- So physics steers it exactly where we want draws.

Adding Momentum

A random push to get things moving

The momentum variable

- We invent an extra variable: the ball's momentum.
- At each step we draw it randomly from a Gaussian.
- It is the kick that starts the ball rolling.

Kinetic energy

- Motion carries kinetic energy — one half the momentum squared.
- It trades off against the potential as the ball moves.
- Uphill, the ball slows; downhill, it speeds up.

The Hamiltonian

Potential plus kinetic, and it barely changes

$$\underbrace{H(\theta, r)}_{\text{total energy}} = \underbrace{U(\theta)}_{\text{potential}} + \underbrace{\frac{1}{2} r^\top r}_{\text{kinetic}} \text{ stays (nearly) constant as the ball rolls}$$

What it is

- The total energy: potential plus kinetic.
- As the ball rolls, energy shifts between the two.
- But the TOTAL stays (almost) constant.

Why that matters

- Constant energy means the proposal is on the right level set.
- So the far-away endpoint is a valid posterior draw.
- That is what makes long moves get accepted.

Why Energy Conservation Helps

Long moves that still get accepted

The random-walk dilemma

- Big proposals move far but are usually rejected.
- Small proposals are accepted but barely move.
- You seem stuck choosing between the two.

How HMC escapes it

- The ball travels far by following the landscape.
- Conserved energy keeps the endpoint just as probable.
- So you get BOTH big moves AND high acceptance.

Following the Gradient

The ball needs to feel the slope

What the ball uses

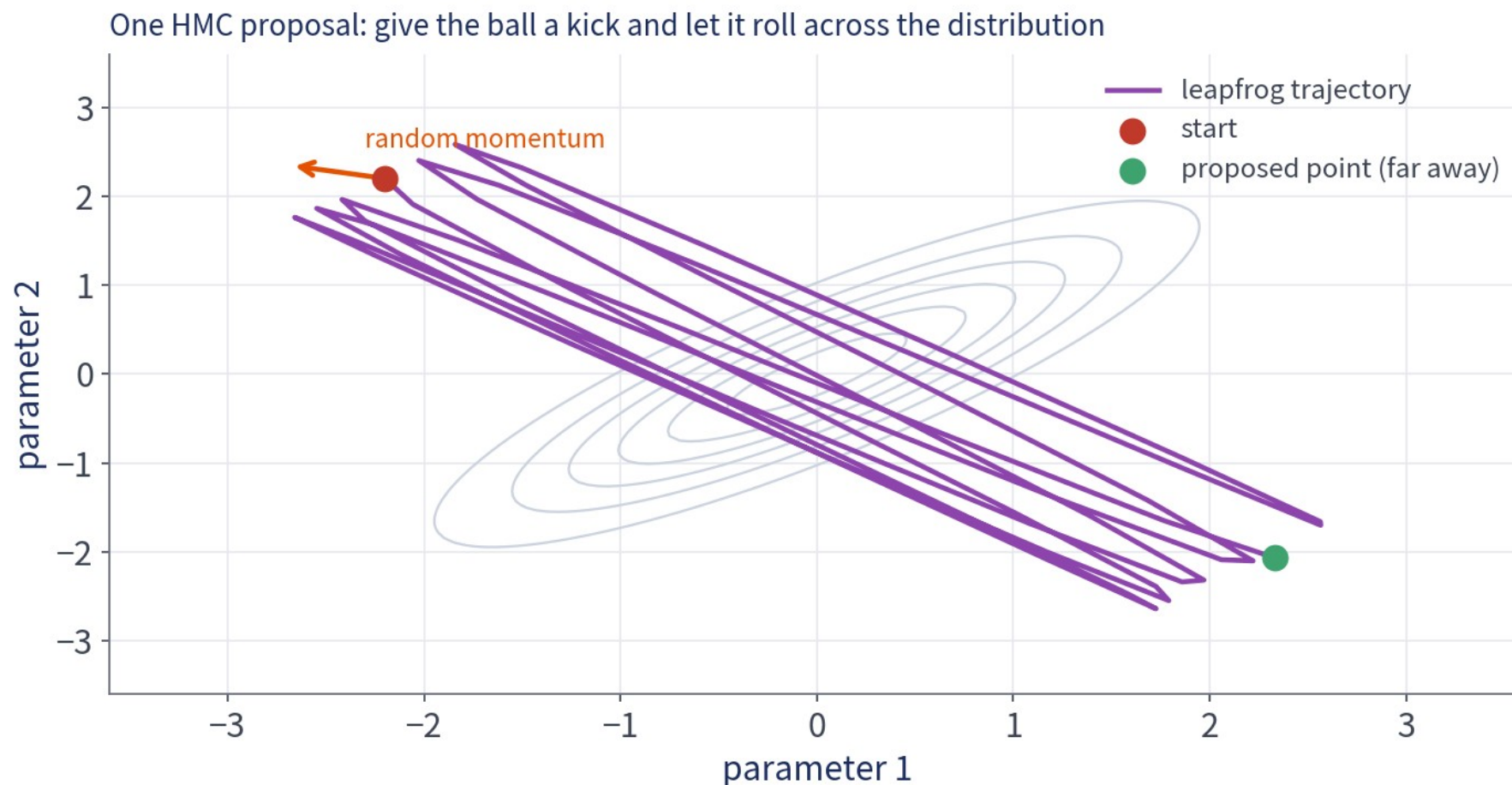
- The slope of the landscape — the gradient.
- It points in the direction the ball should roll.
- This is what makes the moves purposeful, not blind.

The catch

- You must be able to compute that gradient.
- So the model has to be differentiable.
- Modern tools get the gradient automatically.

One HMC Proposal Travels Far

Give the ball a random momentum (orange) and let it roll along the landscape. A single trajectory sweeps clear across the distribution, ending at a distant point that is still a valid posterior draw.



Leapfrog Integration

How the computer rolls the ball

A computer can't roll the ball continuously — it takes small steps, just like discretizing an SDE. The leapfrog method does this in a special order that keeps energy almost perfectly conserved, which is exactly what HMC needs.

The idea

- Advance momentum and position in tiny steps.
- Interleave them: half a momentum step, a full position step, half again.
- Repeat for a chosen number of steps.

Why this order

- It is reversible and volume-preserving.
- So it tracks the true motion with tiny energy drift.
- That near-constant energy keeps acceptance high.

The Leapfrog Step

Half-kick, drift, half-kick

$$\text{leapfrog: } r \leftarrow r - \frac{\varepsilon}{2} \nabla U(\theta); \quad \theta \leftarrow \theta + \varepsilon r; \quad r \leftarrow r - \frac{\varepsilon}{2} \nabla U(\theta)$$

Reading it

- First a half step of momentum, using the slope.
- Then a full step of position, carried by momentum.
- Then another half step of momentum.

The two dials

- The step size sets how fine each move is.
- The number of steps sets how far the ball rolls.
- Tuning them by hand is the awkward part — NUTS fixes it.

The Accept Step

A small correction for a small error

Why we still accept-reject

- Leapfrog is excellent but not perfect.
- The energy drifts by a tiny amount.
- A Metropolis step corrects for that drift.

What it guarantees

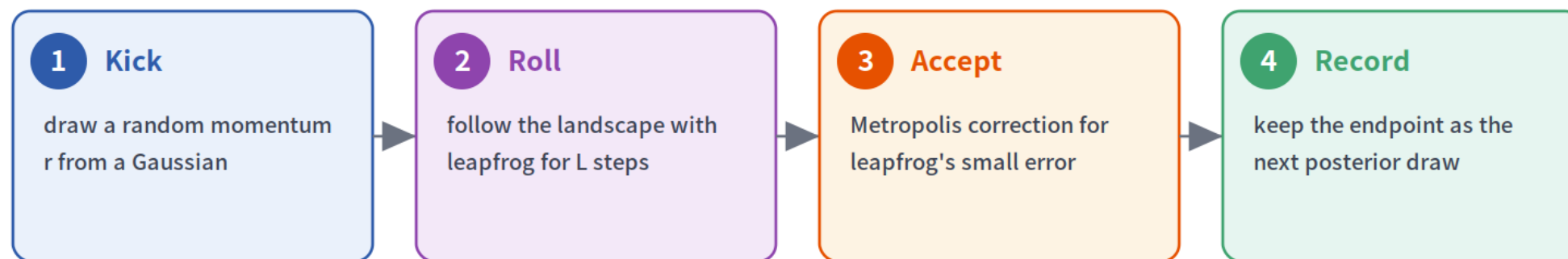
- The chain still targets the exact posterior.
- Because energy barely changes, acceptance is high.
- Rejections are rare, unlike in a random walk.

One HMC Step, End to End

Kick the ball with a random momentum, roll it with leapfrog for several steps, apply a Metropolis correction, and record the endpoint. Because energy is conserved, that distant point is almost always accepted

One HMC Step

Kick the ball, let it roll, then accept - and you have travelled far in one move.



Why this beats a random walk

Because the ball follows the landscape, one roll carries it a long way to a NEW high-posterior region - and because energy is nearly conserved, that far-away proposal is almost always accepted.

Long, purposeful moves with high acceptance - the opposite of aimless wandering.

How Long Should the Ball Roll?

The one dial HMC can't set itself

HMC has a weakness: you must choose how far to roll. Too short, and it behaves like a random walk again. Too long, and the ball loops around and comes back where it started — wasted effort. Getting this right by hand is fiddly.

Too short a roll

- The ball barely moves from where it started.
- You lose the whole advantage of HMC.
- It degenerates into a slow random walk.

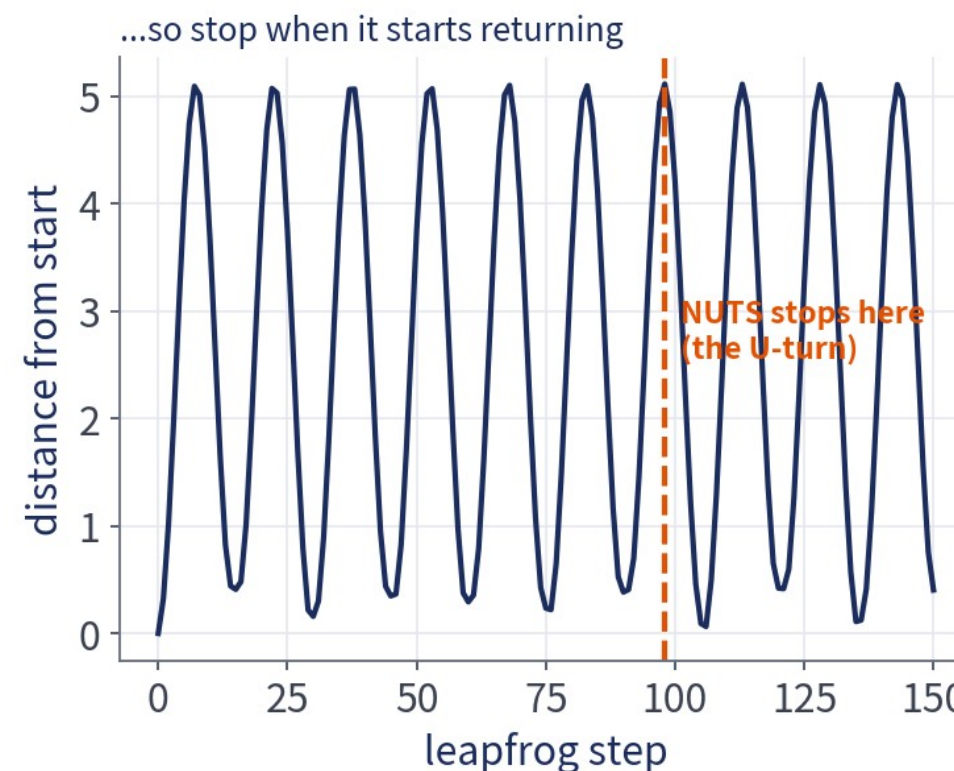
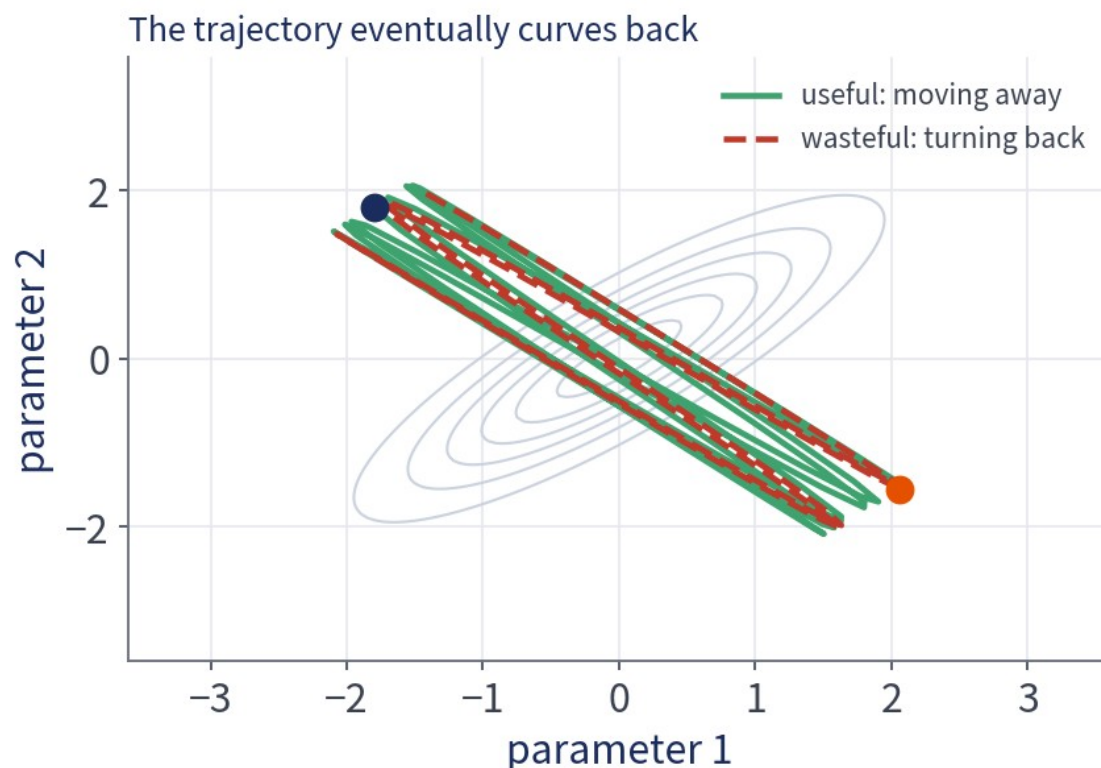
Too long a roll

- The ball curves around and heads back.
- The extra steps are wasted computation.
- It can even return near its starting point.

The U-Turn Problem

As the ball rolls, its distance from the start grows, then peaks, then shrinks as the trajectory curves back. That peak — the moment it begins to return — is the natural place to stop.

How far to roll? NUTS stops automatically at the U-turn



NUTS: Stop at the U-Turn

The No-U-Turn Sampler tunes the length for you

The No-U-Turn Sampler, NUTS, removes the guesswork. It keeps rolling the ball and simply stops the moment the trajectory starts to turn back on itself. No trajectory length to set — it finds the right one automatically, every step.

What NUTS does

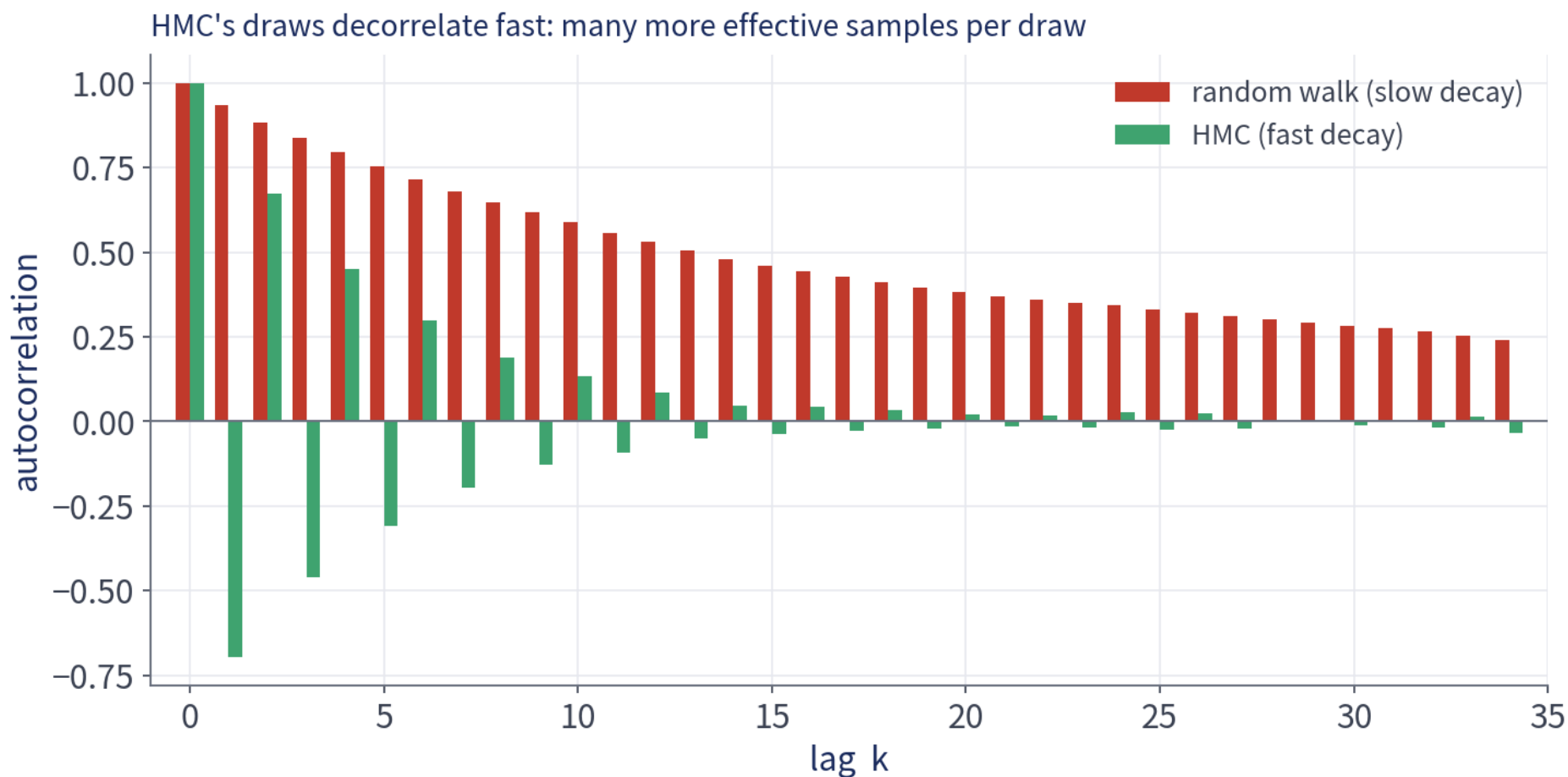
- Extend the trajectory until it starts to U-turn.
- Then stop — no manual length needed.
- It adapts the length at every iteration.

Why it is the default

- It combines HMC's speed with zero length-tuning.
- It just works across a huge range of models.
- It is the engine inside modern Bayesian tools.

Why It Wins: Fast Decorrelation

The random walk's draws stay correlated for many lags; HMC's decorrelate almost immediately. Fewer correlated draws means far more effective samples for the same effort — the practical payoff of rolling smartly.



This Is What PyMC and Stan Use

You get HMC without coding it

What you write

- Just the model: priors and a likelihood.
- The tool computes gradients for you.
- It runs NUTS and tunes the step and length.

What you get

- Efficient, low-correlation posterior draws.
- Far higher effective sample size per second.
- State-of-the-art sampling with almost no effort.

Roll the Ball Yourself

Direct Claude Code to build a tiny HMC

GOAL Code a minimal HMC and see it beat a random walk on the same target.

[INPUT] Prompt · Claude Code

Write and run `hmc_demo.py` that:

1. defines a 2D Gaussian target and its potential $U = -\log \text{posterior}$, plus its gradient;
2. implements leapfrog and a Hamiltonian Monte Carlo step (kick, roll L steps, accept);
3. also runs a random-walk Metropolis sampler on the same target;
4. compares them: overlay the sample paths and plot autocorrelation for each.

Show HMC decorrelates far faster and report each sampler's effective sample size.

How to work with it

- Potential $U = \text{minus log posterior}$.
- Leapfrog: half-kick, drift, half-kick.
- One roll should travel far.
- HMC's ESS should dwarf the random walk's.

Deliverable: `hmc_demo.py` + overlaid sample paths and autocorrelation for HMC vs random walk.

Three Questions Before You Finish

If you can answer these, Part 3 has landed

Q1 In HMC, what plays the role of the landscape the ball rolls on?

Negative log posterior

Q2 Why do HMC's long proposals still get accepted?

Energy is nearly conserved

Q3 What does NUTS decide automatically?

How far to roll (trajectory length)

All three come from the potential landscape, energy conservation, and the U-turn rule.

Three Misconceptions to Avoid

Where HMC intuition slips

Common myths

- "HMC is a totally different idea from Metropolis."
- "HMC works on any model, no matter what."
- "A longer trajectory is always better."

The reality

- It still ends with a Metropolis accept step.
- It needs a differentiable model to get the gradient.
- Too long U-turns and wastes effort — that's why NUTS exists.

Week 14 in Three Ideas

Pulling the whole week together

1

Sample, don't integrate

Parts 1-2: MCMC sidesteps the intractable normalizer by drawing samples.

Metropolis-Hastings proposes and accepts by a ratio; the histogram of draws IS the posterior.

2

HMC rolls instead of wandering

Part 3: treat the parameter as a ball on the negative-log-posterior landscape. A random kick plus a leapfrog roll carries it far in a useful direction, and conserved energy keeps acceptance high.

3

NUTS makes it automatic

The one dial HMC can't set — how far to roll — is handled by NUTS, which stops at the U-turn. This is why PyMC and Stan sample so efficiently with almost no tuning.

Next — Week 15: Bayesian state-space models and HMM, where sampling meets hidden states.

★ WEEK 14 · PART 3 · PRACTICE

HMC and NUTS, Worked by Hand

Roll the Ball One Step, Then Let NUTS Stop It

The Hamiltonian, a leapfrog step, and the U-turn

Part 3's physics-powered sampler, with the arithmetic shown. We build the potential from the posterior, add momentum, and take one leapfrog step - watching the energy stay nearly conserved - then follow the ball as it swings and see NUTS end the roll at the U-turn.

3P

PART 3 · APPLICATIONS

Roll a Ball, Don't Wander

Case A: the Hamiltonian and one leapfrog step - potential plus kinetic energy, and how leapfrog keeps it nearly conserved so the move is almost always accepted. Case B: how far to roll, and how NUTS stops at the U-turn. Then a Claude Code lab.

What Part 3 Told Us

Now we roll the ball - with numbers

1

Random walks waste effort

Metropolis wanders aimlessly. HMC instead rolls a ball down the posterior's landscape, moving far in the right direction.

2

Energy guides the motion

The potential is the negative log posterior; momentum is a random push. Their sum, the Hamiltonian, barely changes as the ball rolls.

3

NUTS sets the one dial for you

How far to roll is the only knob HMC can't set. NUTS rolls until the ball turns back on itself, then stops.

Everything here is a calculator exercise: one leapfrog step, one energy check, one U-turn.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from a position and a momentum.
- Roll with leapfrog, or follow the swing.
- Then read the energy or the U-turn.

What you should watch

- How the gradient steers the ball.
- How the total energy stays nearly flat.
- How the ball turns back on itself.

One Roll of the Ball

The Hamiltonian, and a leapfrog step

HMC turns the posterior into a landscape: the potential energy is its negative log, so high-probability regions are valleys. Add a random momentum, and the ball rolls. One leapfrog step - half-kick, drift, half-kick - moves it, and the total energy barely changes, so the move is almost always accepted. We take that step by hand

The setup

- A standardized Gaussian posterior.
- Potential $U = \text{half } \theta^2$.
- Start at $\theta = 1$, momentum = 1.

What we'll do by hand

- Take one leapfrog step, size 0.3.
- Check the total energy before and after.
- Compute the acceptance probability.

Potential, Gradient, Hamiltonian

The landscape the ball rolls on

$$\underbrace{U(\theta) = -\log p(\theta \mid D)}_{\text{the landscape}}, \quad \nabla U = \theta, \quad \underbrace{H = U(\theta) + \frac{1}{2}p^2}_{\text{potential} + \text{kinetic}}$$

The potential

- It's the negative log posterior.
- Valleys are high-probability regions.
- Its gradient is the slope the ball feels.

The Hamiltonian

- Potential plus kinetic energy.
- The kinetic part is half momentum-squared.
- It stays nearly constant as the ball rolls.

One Leapfrog Step

Half-kick, drift, half-kick

$$\text{half-kick: } p \leftarrow p - \frac{\epsilon}{2} \nabla U = 1.0 - 0.15 = 0.85$$

$$\text{drift: } \theta \leftarrow \theta + \epsilon p = 1.0 + 0.3(0.85) = 1.255$$

$$\text{half-kick: } p \leftarrow 0.85 - 0.15(1.255) = 0.662$$

The momentum gets a half-kick from the gradient, the position drifts with that momentum, then a final half-kick completes the step - moving from $\theta=1.0$ to 1.255. The gradient did the steering.

Energy Nearly Conserved

A tiny error, a tiny correction

$$H : 1.000 \rightarrow 1.0065 \ (\Delta H = +0.0065) \Rightarrow \text{accept} = \min(1, e^{-\Delta H}) = \mathbf{0.994} \text{ (energy nearly conserved)}$$

The check

- The Hamiltonian went from 1.000 to 1.0065.
- A change of just 0.0065.
- Leapfrog keeps energy almost exact.

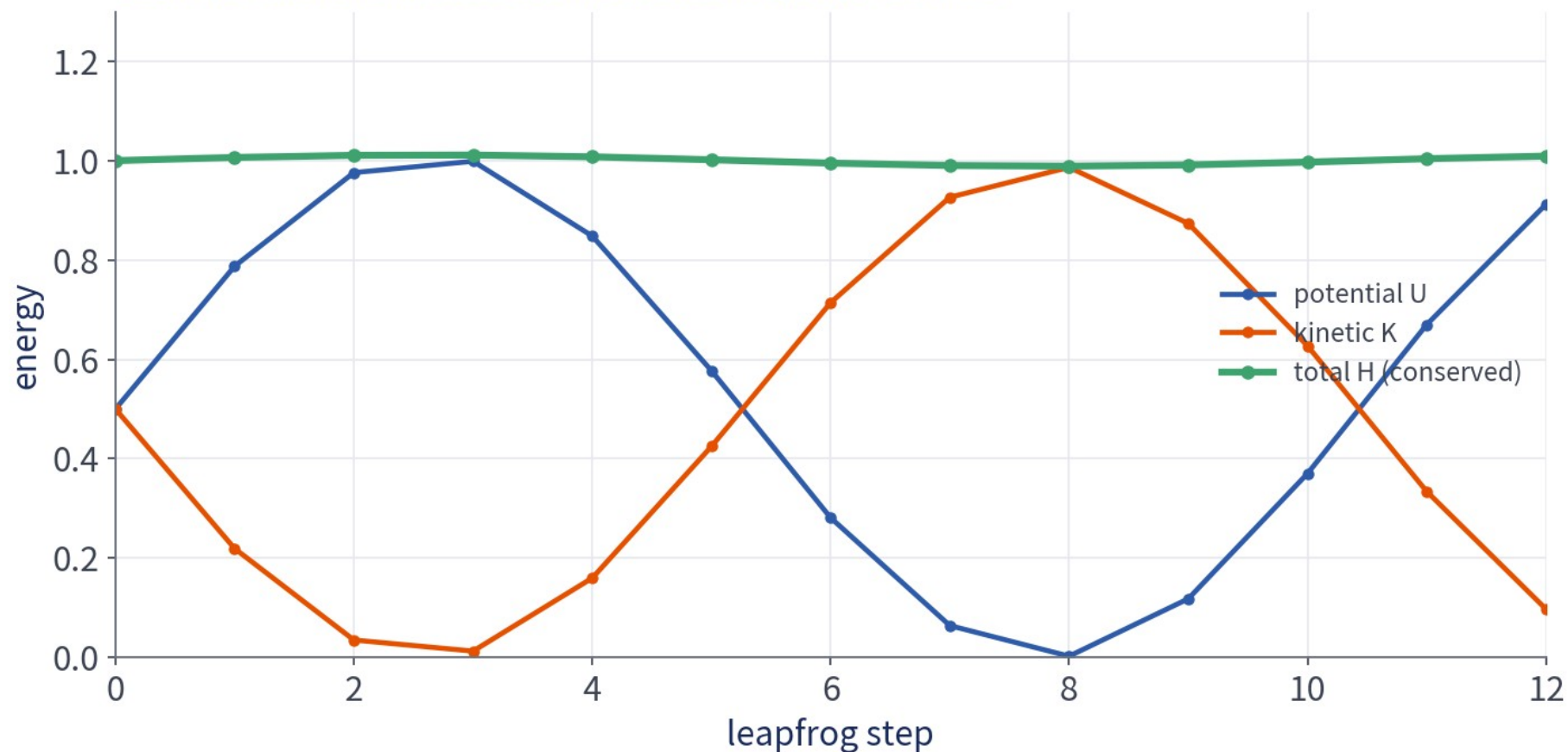
The accept step

- Accept with e-to-the-minus-that-change.
- That's 0.994 - almost always yes.
- The correction fixes leapfrog's small error.

Energy Trades, But Total Holds

As the ball rolls, potential and kinetic energy swap back and forth - the ball speeds up in the valley and slows on the rise. But their sum, the total energy, stays nearly flat across every leapfrog step. That near-conservation is why HMC accepts almost every move

Case A: potential and kinetic trade off, but total energy stays nearly flat



What Posterior Is the Ball On?

A strategy's expected return μ is a real finance posterior in disguise. Suppose a strategy's expected annual return has posterior $N(10\%, 5\%)$. Standardize it - z is the number of posterior standard deviations from the mean - and the potential becomes exactly half z -squared. The ball's position is a plausible

$$\text{posterior } \mu \sim N(10\%, 5\%), \quad z = \frac{\mu - 10\%}{5\%} : \quad z=1.0 \leftrightarrow \mu=15\%, \quad z=1.255 \leftrightarrow \mu=16.3\%$$

So the leapfrog move from $z=1.0$ to 1.255 is the sampler exploring expected returns from 15% up to 16.3%. The gradient always points back toward 10% - the posterior's most probable return.

Why the Ball Beats the Walk

Direction, not luck

The random walk

- Proposes blindly in every direction.
- Rejects most far moves.
- Crawls through the posterior.

The rolling ball

- Uses the gradient to move purposefully.
- Travels far and still gets accepted.
- Explores in a fraction of the steps.

How Far to Roll?

The one dial, and how NUTS sets it

HMC's only real dial is how long to let the ball roll. Too short, and it barely moves - a random walk again. Too long, and the ball swings out and comes right back, wasting the whole trajectory. NUTS watches the ball and stops the moment it starts turning back toward the start. We trace the swing and find that 11-turn

The trajectory

- The ball swings through the valley.
- It reaches a far point, then returns.
- Same energy, position oscillating.

What we'll do by hand

- Follow the position over the steps.
- Find where it starts moving back.
- That's where NUTS stops.

The Trajectory-Length Dial

Too short, too long, or just right

roll too short $\Rightarrow$ random walk; too long $\Rightarrow$ U-turn back (wasted); tune the length $L = \epsilon \times (\text{\#steps})$

The two failures

- Too short: tiny move, like a random walk.
- Too long: the ball U-turns back to the start.
- Both waste the gradient information.

The length

- It's the step size times the number of steps.
- Tuning it by hand is fiddly and problem-specific.
- NUTS removes the need entirely.

The Ball Swings

Position over the leapfrog steps

step	0	3	5	7	9
theta	+1.00	+1.41	+1.07	+0.36	-0.49
momentum	+1.00	-0.16	-0.92	-1.36	-1.32

The ball climbs to its farthest point near step 3, where the momentum flips negative - it has started coming back. From there it swings through the valley and out the other side. Positive position with negative momentum is the U-turn signal NUTS watches for.

By step 3 the ball has turned around: NUTS ends the roll at this U-turn.

NUTS Stops at the U-Turn

No length to tune

NUTS stops when $(\theta - \theta_0) \cdot p < 0$ (the ball turns back) $\Rightarrow$ no length to tune

The U-turn test

- Watch the displacement times the momentum.
- While positive, the ball moves away.
- When negative, it's heading back.

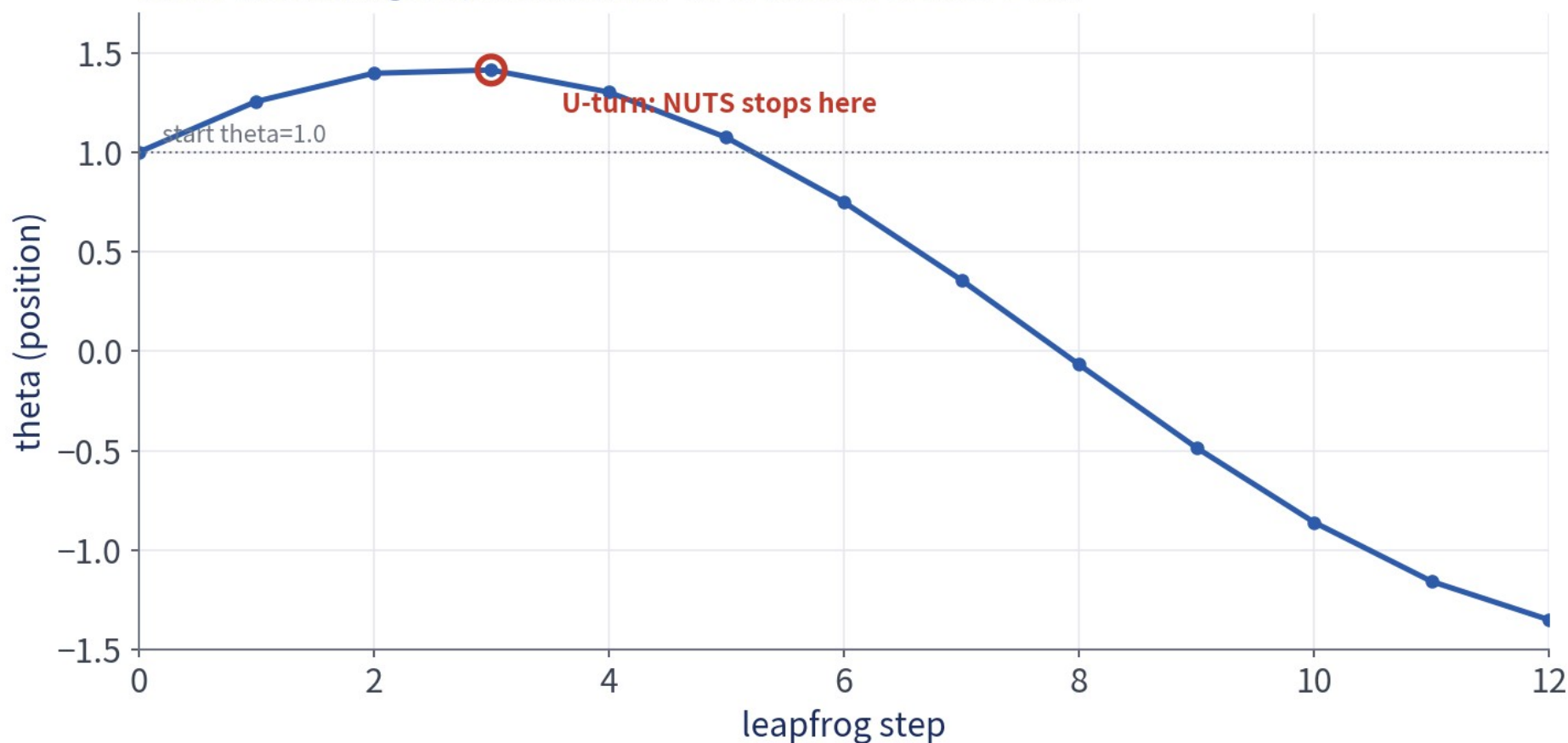
The payoff

- NUTS ends the roll right there.
- No trajectory length to set by hand.
- The longest useful move, automatically.

Out, Then Back - Stop at the Turn

The ball's position climbs to a peak near step 3, then swings back down through the start and out the far side. The circled point is the U-turn, where it first heads back toward where it began - exactly where NUTS ends the trajectory, capturing the longest useful move

Case B: the ball swings out, then turns back - NUTS ends the roll at the U-turn

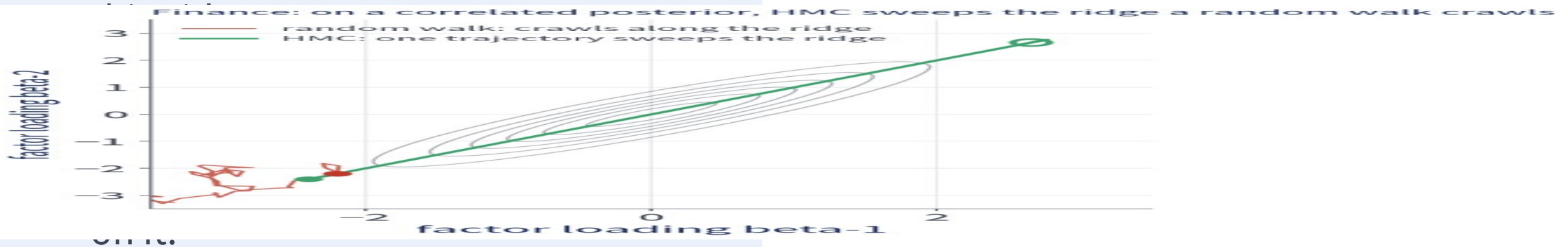


Why Finance Needs the Roll

Correlated parameters, an elongated posterior

The finance posterior

- Two overlapping factor loadings.
- Posterior correlation 0.9 - a long,



- One HMC trajectory (green) sweeps the ridge,
- then U-turns at the far end.

HMC You Don't Have to Tune

Why PyMC and Stan use NUTS

What NUTS gives you

- Automatic trajectory length, per step.
- No manual tuning of how far to roll.
- Long, efficient, accepted moves.

Why it's the default

- PyMC and Stan run NUTS under the hood.
- You write the model, it does the sampling.
- Gradient-based efficiency, no hand-tuning.

Why HMC Wins

Gradients allow moves without tuning

HMC replaces aimless wandering with directed motion. The posterior becomes a landscape, momentum sets the ball rolling, and leapfrog follows the gradient while conserving energy - so far-reaching moves are almost always accepted. NUTS then sets the one remaining dial by stopping at the U-turn. That combination is

The roll (Case A)

- Potential from the posterior, plus momentum.
- Leapfrog conserves energy.
- Acceptance near one.

The length (Case B)

- The ball swings and turns back.
- NUTS stops at the U-turn.
- No tuning, longest useful move.

Reproduce the Roll and the U-Turn

Direct Claude Code to check your arithmetic

GOAL Verify one leapfrog step, the energy conservation, and the U-turn detection.

[INPUT] Prompt · Claude Code

Write and run `part3_practice.py` that, for $U=0.5*\theta^2$ with gradient θ and step size 0.3:

CASE A - one step:

- * from $\theta=1$, $p=1$, do one leapfrog step;
print $\theta=1.255$, $p=0.662$, and H before/after;
- * print the acceptance probability e^{-dH} .

CASE B - the swing:

- * run 12 leapfrog steps, print θ and p each step;
- * flag the first U-turn where $(\theta-\theta_0)*p < 0$.

Then plot the energy and the position trajectory.

Match every printed number to the slides.

Check against the slides

- leapfrog: θ 1.255, p 0.662.
- H 1.000 $\rightarrow$ 1.0065.
- accept 0.994.
- U-turn near step 3.

Deliverable: `part3_practice.py` whose printed numbers match the leapfrog step and the U-turn.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What is the potential energy in HMC?

The negative log posterior - the landscape

Q2 Why is HMC's acceptance so high?

Leapfrog nearly conserves the total energy

Q3 How does NUTS set the trajectory length?

It stops when the ball starts turning back

All three come from the Hamiltonian roll, the energy check, and the U-turn rule.

Three Pitfalls to Avoid

Where HMC trips people

Common mistakes

- "HMC doesn't need the gradient."
- "A longer roll is always better."
- "You have to code HMC yourself."

The reality

- The gradient is essential - it steers the ball.
- Too long and the ball U-turns, wasting the move.
- PyMC and Stan run NUTS for you.

Two Calculations to Remember

Part 3 applications in one screen

1

One leapfrog step, energy conserved

From the $\theta=1$ and momentum = 1, a leapfrog step of size 0.3 moved the ball to $\theta=1.255$, and the total energy shifted only from 1.000 to 1.0065 - so the move is accepted with probability 0.994. The gradient steers, and leapfrog keeps energy nearly exact.

2

The ball swings, and NUTS stops it

Following the trajectory, the ball reached its farthest point near step 3, where the momentum flipped negative - the U-turn. NUTS ends the roll there, capturing the longest useful move with no trajectory length to tune by hand.

3

Directed motion, automatically tuned

HMC rolls a ball down the posterior using the gradient, conserving energy for high acceptance; NUTS sets the one dial by stopping at the U-turn. That is exactly what PyMC and Stan run for you.

Week 14 complete — from sampling instead of integrating, through Metropolis and Gibbs, to gradient-powered HMC and NUTS.